

LLM-Powered Static Binary Taint Analysis

PUZHUO LIU, Ant Group, Hangzhou, China and Tsinghua University, Beijing, China
CHENGNIAN SUN, University of Waterloo, Waterloo, Ontario, Canada
YAOWEN ZHENG, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China
XUAN FENG, Independent Researcher, Vancouver, British Columbia, Canada
CHUAN QIN and YUNCHENG WANG, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China
ZHENYANG XU, University of Waterloo, Waterloo, Ontario, Canada
ZHI LI, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China
PENG DI, Ant Group, Hangzhou, China
YU JIANG, Tsinghua University, Beijing, China
LIMIN SUN, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China

This article proposes LATTE, the first static binary taint analysis that is powered by a large language model (LLM). LATTE is superior to the state of the art (e.g., Emtaint, Arbiter, Karonte) in three aspects. First, LATTE is fully automated while prior static binary taint analyzers need rely on human expertise to manually customize taint propagation rules and vulnerability inspection rules. Second, LATTE is significantly effective in vulnerability detection, demonstrated by our comprehensive evaluations. For example, LATTE has found 37 new bugs in real-world firmware, which the baselines failed to find. Moreover, 10 of them have been assigned CVE numbers. Lastly, LATTE incurs remarkably low engineering cost, making it a cost-efficient and scalable solution for security researchers and practitioners. We strongly believe that LATTE opens up a new direction to harness the recent advance in LLMs to improve vulnerability analysis for binary programs.

This work was supported by the Ant Group Postdoctoral Programme.
Authors' Contact Information: Puzhuo Liu (corresponding author), Ant Group, Hangzhou, China and Tsinghua University, Beijing, China; e-mail: liupuzhuo.lpz@antgroup.com; Chengnian Sun, University of Waterloo, Waterloo, Ontario, Canada; e-mail: cnsun@uwaterloo.ca; Yaowen Zheng, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China; e-mail: zhengyaowen@iie.ac.cn; Xuan Feng, Independent Researcher, Vancouver, British Columbia, Canada; e-mail: xfeng9209@gmail.com; Chuan Qin, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China; e-mail: qinchuan@iie.ac.cn; Yuncheng Wang, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China; e-mail: wangyuncheng@iie.ac.cn; Zhenyang Xu, University of Waterloo, Waterloo, Ontario, Canada; e-mail: zhenyang.xu@uwaterloo.ca; Zhi Li, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China; e-mail: lizhi@iie.ac.cn; Peng Di (corresponding author), Ant Group, Hangzhou, China; e-mail: dipeng.dp@antgroup.com; Yu Jiang, Tsinghua University, Beijing, China; e-mail: jy1989@mail.tsinghua.edu.cn; Limin Sun, Institute of Information Engineering CAS, Beijing, China and School of Cyber Security, University of the Chinese Academy of Sciences, Beijing, China; e-mail: sunlimin@iie.ac.cn.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7392/2025/2-ART83
https://doi.org/10.1145/3711816

Downloaded from the ACM Digital Library on April 7, 2025.

CCS Concepts: • **Security and privacy** → **Software security engineering**; • **Software and its engineering** → *Software post-development issues*;

Additional Key Words and Phrases: binary, taint analysis, large language model, vulnerability

ACM Reference format:
Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhenyang Xu, Zhi Li, Peng Di, Yu Jiang, and Limin Sun. 2025. LLM-Powered Static Binary Taint Analysis. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 83 (February 2025), 36 pages.
https://doi.org/10.1145/3711816

1 Introduction

For decades, software testing has remained the dominant method for software quality assurance, primarily due to its cost-effectiveness. However, as Dijkstra emphasized [19], software testing can only reveal the presence of bugs, not their absence. Consequently, it is not uncommon to encounter bugs in released binaries [78, 88]. Certain bugs can be particularly critical and may be exploited as vulnerabilities to initiate cyberattacks, e.g., unauthorized access to systems, stealing sensitive data, and disrupting normal system operations [4, 41].

To reduce the impact of vulnerabilities in released binaries such as firmware of embedded devices [12, 13, 74, 103] and commercial software applications [7, 33, 67, 88], third-party security audits emerged as an indispensable force. These audits, conducted by entities without access to the source code, apply various post-development analysis [13, 52, 71, 102] and testing techniques [12, 74, 88, 103] on the binaries. *Thus, though analysis and testing are usually applied on source code, analyzing and testing binaries is also an indispensable, complementary technique to ensure software reliability and security.*

Static Binary Taint Analysis. Among these analysis and testing techniques [12, 13, 103], static binary taint analysis (referred to as *taint analysis* for brevity in this article) stands out as an effective approach for vulnerability inspection. It can directly analyze binaries with several advantages, including high test code coverage, independence from the concrete execution environment, and the ability to discover previously unknown exploitable vulnerabilities [12, 74, 88].

The standard taint analysis workflow comprises three steps. ① *Identifying Taint Sources*: Taint sources are typically functions that receive external data. These external data can be manipulated by attackers. For example, the function `recv` reads data through a socket, `getenv` retrieves information from an environment variable, and `fgets` loads data from a file. ② *Propagating Taint Labels*: Initially, the external data received through taint sources are labeled tainted. Then the analysis traces the flow of the tainted data through the program; it propagates the taint labels along data dependencies, such as assignments, computations, and function calls, to discover how the tainted data spread throughout the program. Meanwhile, if a memory region with a taint label is overwritten with untainted data or the region is sanitized with certain checks, then the taint label is removed from that memory region. ③ *Identifying and Inspecting Sinks*: Sinks are potential points in the program where tainted data might be misused or lead to security vulnerabilities. They are usually sensitive operations including manipulating strings (e.g., `strcpy`, `strcat`), executing commands (e.g., `system`, `execl`), and formatted outputting (e.g., `printf`, `fprintf`).

Limitations of Taint Analysis. Taint analysis remains a partially automated process, demanding considerable human effort for its execution. The effectiveness of this analysis heavily relies on the accuracy and quality of rules governing taint source, taint label propagation, and sink inspection. In particular, using taint analysis for vulnerability detection not only involves the effective propagation of taints but also requires setting up as complete taint checking rules as possible to reduce

Downloaded from the ACM Digital Library on April 7, 2025.

False Positives (FPs) and **False Negatives (FNs)**. Regrettably, designing these rules necessitates meticulous manual crafting, resulting in a non-trivial and error-prone task. Previous techniques [11, 12, 31, 45, 74, 88] have relied on human expertise to manually define these rules, thereby introducing the risk of FPs or FNs in the analysis results due to potential rule errors and absences [10, 45]. For instance, determining taint sources involves identifying functions that receive external data, and establishing the initial taint labels requires the specification of these functions. Additionally, the sanitization checks that transform tainted data into safe data in propagation rules are diverse and specific to function semantics. Furthermore, the inspection rules also vary based on the types of vulnerabilities and sinks involved (further details in Section 3).

Insights. The limitation of taint analysis primarily arises from the requirement of an in-depth understanding of the semantics of the code and the vulnerability pattern. Extension of rules has historically imposed a significant burden on humans when applying taint analysis to different binaries or vulnerabilities [11, 52, 88]. To overcome this limitation, we firmly believe that **Large Language Models (LLMs)** can provide valuable assistance, benefiting from their recent advances in code understanding, including code generation [66, 96, 99] and code summarization [3, 82]. LLMs have advantages in code-related tasks by providing context awareness, semantic understanding, and natural language interaction. While LLMs have their advantages, it is essential to note that they are not a silver bullet. Therefore, in this article, we embark on the first study for vulnerability discovery that explores the synergistic combination of the ability of LLM in code understanding and the concept of taint analysis. This will improve the automation and accuracy of taint-style vulnerability inspections.

Challenge of Using LLMs. There are unexplored challenges in applying LLMs for vulnerability inspection in binaries. ① Prompt engineering: Prompts play a crucial role in instructing LLMs to generate responses, and their design significantly influences the performance of LLMs. In the context of vulnerability analysis, well-constructed prompts must include the code snippets to be analyzed and a clear description of the analysis tasks. However, current LLMs have their limitations: Providing all the code snippets related to potential vulnerabilities at once or directly instructing vulnerability analysis often leads to unsatisfactory results. Therefore, it is preferable to break down the relevant code snippets and vulnerability analysis tasks, and prompt LLMs to perform step-by-step analysis. Furthermore, the prompt engineering process should be designed to be general, without requiring human intervention whenever the program or vulnerability analysis task changes. ② Encoding of code context: To conduct accurate vulnerability analysis, having adequate code context, including function call stacks, data dependencies, and control flow, is crucial to determine whether a piece of code is vulnerable. However, most LLMs currently accept only a limited amount of context, striking a balance between computational costs and response quality. For example, the state-of-the-art LLM GPT-4.0 supports contexts consisting of just 32,000 tokens (approximately 24,000 words) for both prompts and responses [64]. Consequently, when dealing with real binaries, encoding the code context to meet this size constraint of LLMs becomes necessary. Analyzing the entire binary directly is often impractical, as the decompiled code itself usually surpasses this size limitation.

LLM-Powered Binary Taint Analyzer (LATTE). We propose LATTE, the first technique to inspect vulnerabilities in released binaries by combining the power of LLMs and the concept of taint analysis. It conquers the practical limitations of taint analysis by harnessing the ability of LLMs in code understanding to automate and streamline the manual components of taint analysis, and addresses the challenges of using LLMs with dedicated static analysis and prompt engineering. The key idea of LATTE is to construct **Prompt Sequences (PSs)** driven by code slicing. Concretely,

Downloaded from the ACM Digital Library on April 7, 2025.

(1) Initially, we preprocess the binary to recover the code structure, by performing analysis such as disassembly and decompilation.
(2) Based on the preprocessing results, we use LLMs to identify sinks and then determine potentially **Vulnerable Destinations (VDs)**, i.e., call sites of security sensitive functions (sinks). These VDs will need to be inspected.
(3) With the identified VDs, we perform *backward* intraprocedural and interprocedural data dependency slicing to establish function **Call Chains (CCs)** starting from the VDs.
(4) We consider function CCs involving external input data as **Dangerous Flows (DFs)**. To this end, we first use LLMs to identify sources that pass externally controllable data into the program and then identify such CCs that include sources.
(5) Based on the DFs, we use functions as units and combine them with program analysis tasks, such as flow analysis, alias analysis, and taint analysis, to form PSs. These PSs are generated to converse with LLMs, completing the vulnerability inspection.

Evaluations. Our comprehensive evaluations demonstrate that LATTE significantly outperforms the state-of-the-art binary taint analyzers. Specifically, We first evaluated LATTE based on the taint-style vulnerability test subjects (compiled and stripped) in the standard dataset Juliet Test Suite [23]. The accuracy and F1 of LATTE vulnerability checking exceeds Emtaint [12] and Arbiter [88] on every vulnerability type. Moreover, LATTE achieved 100% correct coverage for identifying sinks and sources in the test subject target scenario. For test subjects labeled with containing vulnerabilities, LATTE successfully extracted more than 95% of the correct DFs that covered the paths triggering potential vulnerabilities. Furthermore, we assessed the effectiveness of LATTE in real-world binaries using the embedded device firmware dataset provided by Karonte [74]. LATTE detected a total of 119 unique bugs, including 37 previously unknown bugs (10 CVE numbers have been given due to high threat), which outperforms the state of the art [12, 74, 88].

Contributions. We make the following major contributions:

—We propose LATTE, the first technique to leverage LLMs to power static binary taint analysis. LATTE addresses the limitations of prior taint analysis. Prior taint analyzers heavily rely on human expertise to define rules for propagating taint labels and inspecting sinks and customize rules for different binaries and vulnerability types, whereas LATTE harnesses the ability of LLMs in code understanding to fully automate taint analysis.
—LATTE presents a novel solution to address the practical challenges of using LLMs, with dedicated multiple-step static analyses and automated prompt engineering to effectively elicit LLMs to generate accurate analysis responses.
—Comprehensive evaluations demonstrate the effectiveness of LATTE. Compared to the state of the art, LATTE achieved higher accuracy and F1 score on the standard dataset, and on the firmware dataset, LATTE found 117 unique bugs (including 37 previously unknown bugs with 10 awarded CVE numbers), at least 21 more than the baselines.

2 Background

2.1 Taint-Style Vulnerability

As an effective software testing technique, taint analysis is often used for binary vulnerability detection [10–12, 31, 45, 73, 74]. Taint analysis can effectively find vulnerabilities caused by external inputs, commonly referred to as taint-style vulnerabilities [98]. Nevertheless, previous binary taint techniques [10–12, 31, 45, 73, 74] use different names to denote types of vulnerabilities they support. To facilitate comparison of different techniques in a unified, standard way, we categorize

Downloaded from the ACM Digital Library on April 7, 2025.

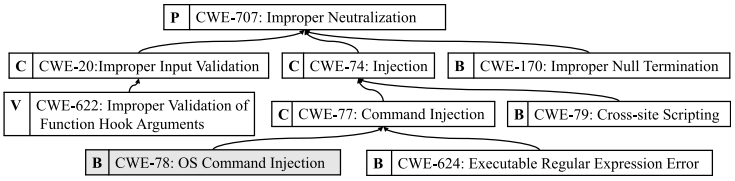


Fig. 1. Example of the class CWE-78.

Table 1. Mapping between Taint-Style Vulnerability Types and CWE, Comparison of Binary Taint Analyzer Capabilities

Vulnerability Type		Binary Taint Analyzer							
		Saluki [31]	Dtaint [11]	Karonte [74]	SATC [10]	FBI [45]	Arbiter [88]	Emtaint [12]	LATTE
CWE-78	OS Command Injection	✓	✓		✓	✓	✓[a]	✓	✓
CWE-120	Classic Buffer Overflow		✓	✓	✓	✓	✓[a]	✓	✓
CWE-134	Controlled Format String					✓	✓	✓	✓
CWE-190	Integer Overflow or Wraparound						✓		✓
CWE-606	Unchecked Loop Condition			✓					✓

"✓" means that the corresponding taint analyzer supports the corresponding vulnerability type.
[a]indicates that the taint analyzer Arbiter does not provide the corresponding inspection rules and additional engineering effort is required.

the vulnerability types supported by previous binary taint analyzers according to the **Common Weakness Enumeration (CWE)** [55].

CWE is a community-developed list of software weakness types used to classify security vulnerabilities [54]. CWE is a tree-like relationship between weaknesses at different levels of abstraction, which can be divided into *pillars*, *classes*, *bases*, and *variants*. As shown in Figure 1, the *pillar* (P) is the weaknesses described in the most abstract way. The *class* (C) is still abstract and usually independent of any particular language or technology. The *basic* (B) is used to present more specific types of weaknesses. The *variant* (V) is described at a low level of details, usually restricted to a specific language or technology.

To ensure that vulnerability types do not overlap, we classify taint-style vulnerability types according to the following principles. ① Vulnerability types are mapped to the CWE's basic level, which is used to present specific types of weaknesses. If basic level does not exist, then the variant level is used. ② Classification is based on the cause the vulnerability is triggered, regardless of the reason for the introduction of tainted data. Otherwise a vulnerability would have multiple labels. For example, if a command injection vulnerability (CWE-78) is caused by unchecked input data, it will be classified as CWE-78, not CWE-76 (Improper Neutralization of Equivalent Special Elements). ③ Classification is based on the first reason the vulnerability is triggered. For instance, if there is an integer overflow vulnerability (CWE-190) in the operation of input data, and the subsequent use of the operation result leads to a buffer overflow (CWE-680: Integer Overflow to Buffer Overflow), we still classify it as CWE-190.

Table 1 lists five CWE vulnerability types, which cover the vulnerability types supported by the previous binary taint analyzer. Previous techniques rely on human expertise or heuristics to define taint analysis rules including taint sources, sanitizers, and sinks, to propagate and inspect tainted data to discover taint-style vulnerabilities in binaries. While theoretically it is possible to apply a well-implemented taint engine and sound rules to inspect all types of vulnerabilities in any binary, this is not realistic.[1]

[1]A taint engine is orthogonal to taint rules. A taint engine is the tool that actually performs taint analysis, detecting potential security vulnerabilities by tracking dataflows. Taint rules are a set of principles and logic that guide the work of the taint engine, defining which data are tainted and how to track and process these data.

Downloaded from the ACM Digital Library on April 7, 2025.

First, implementing a taint engine demands substantial engineering effort, particularly for analyzing binary code. Binaries often lack information on symbols and data types, and can vary significantly due to differences in architecture, compilers, and optimization options. Even in decompiled code, much semantic and structural information is lost or ambiguous. Consequently, these previous techniques typically have to make certain assumptions to develop new taint engines [11, 12, 31, 74, 88].

Second, defining sound taint source, sanitization, and sink rules for existing taint engines is also a complex engineering task. Methods such as function abstraction and machine learning can assist in setting relevant rules for new vulnerability types and libraries [10, 45, 46]. However, to improve the accuracy of vulnerability analysis and reduce the workload of post-analysis, the complexity of the rules and the demand for automatically verifying vulnerabilities inevitably increase. For example, to reduce FPs, certain taint engines [12, 74] perform vulnerability inspections at the sinks of `strcpy`, `strncpy`, and `memcpy` by comparing the length of the taint source and the size of the destination buffer; these rules increase the analysis complexity and workload, and demand more insights from expert experience.

2.2 LLM

An LLM consists of billions of parameters and is trained on billions of samples to be highly general. Downstream tasks can be accomplished using LLMs through either prompt engineering [17, 38, 42, 97] or fine-tuning [18]. In the context of prompt engineering, the downstream task is directly provided as a natural language description to the LLM, prompting the LLM to generate a response to complete the task. Unlike fine-tuning, which involves retraining the LLM with additional data, prompt engineering does not necessitate extra training data and avoids the high computational resource costs. Moreover, the LLM has been trained on extensive corpora and possess the ability to execute specific tasks with prompt engineering [6, 32, 43, 51, 97]. Therefore, we chose the prompt engineering method to conduct our research.

Prompt engineering is a critical process that involves constructing prompts to elicit desired responses from LLMs. It plays a direct role in influencing the performance of LLMs across various tasks. With deliberated prompt engineering, LLMs have demonstrated excellent code understanding capabilities on many code-related tasks, such as code generation [66, 96, 99] and code summarization [3, 82]. These successes inspired us to explore extending the power of LLMs into the field of vulnerability analysis.

3 Motivation

Benefiting from the rich code information and community resources, the development of source code vulnerability static analysis tools has far surpassed that of binary static analysis tools [5, 20, 27]. Compared with source code, in binary, even decompiled pseudocode, there is still a lack of clear functions, classes, variable names, and comments, which makes it difficult to understand specific functions. Table 1 demonstrates that each prior binary taint technique only supports a limited set of taint-style vulnerability types. This limitation arises from the difficulty in manually defining comprehensive propagation and inspection rules for taint analysis. Efforts such as SATC [10] and FBI [45] have attempted to optimize Karonte's [74] rules using human expertise and heuristics, resulting in reduced FNs and FPs. However, due to the complexity and variety of taint analysis, the manual-dependent nature still hinders the overall performance of these binary taint analyzers. As shown in Figure 2, we take the integer overflow vulnerability in a **Decompiled Binary (DB)** program as an example to intuitively illustrate the following concrete problems of designing and implementing effective binary taint analyzer.

Downloaded from the ACM Digital Library on April 7, 2025.

```
1  void foo(void) {
2    signed char a, b, c, d;
3    int e;
4    fscanf(stdin, "%d", &a);
5    b = a;
6    c = 0;
7    a = 1;
8    e = b + 1;  //No integer overflow occurs after type conversion
9    printf("%d",(ulong)e);
10   if (b == 127){
11     printLine("Data value is too large");
12   }
13   else{
14     c = b + 1;   //No integer overflow occurs after sanitization
15     printf("Result is %d\n", c);
16   }
17   d = c + 1;  //The input is 126 causing an integer overflow
18   printf("Result is %d\n", d);
19   return;
20 }
```

Fig. 2. Example of vulnerabilities in decompiled code (comments are added to aid understanding). When the user input from stdin is 126, the expected result on line 17 is 128 (the actual result is −128). Because the char type is signed, its range is [−128, 127]. When the result exceeds this range, an integer overflow occurs.

3.1 Problem 1. Determining Taint Sources and Initial Taint Labels

Manually identifying taint sources, which receive external input in a binary, is a laborious process, because not only standard C/C++ functions (such as `recv`, `fscanf`, `fgets`) but also third-party functions (e.g., `SSL_read` and `BIO_read` in OpenSSL) can receive external data. More importantly, determining the initial taint labels for the return values and parameters of the taint sources requires deep understanding of the semantics of the taint sources. For example, only with the semantics of `fscanf` can we determine the `third` parameter `&a` of `fscanf` (line 4) to be tainted, as `&a` stores the external input retrieved by `fscanf`; the first and second parameters should not be labeled as tainted. Note that it is even more challenging to determine initial taint labels for third-party functions than standard C/C++ functions because the former usually do not have documentation whereas the latter are at least documented in the C/C++ language standard. Mislabeling often leads to the incorrect propagation of taint information throughout the program under analysis.

3.2 Problem 2: Defining Rules to Propagate Taint Labels

Initial taint labels are propagated throughout the program with assignments (e.g., `b = a` on line 5), calculations (e.g., `e = b+1` on line 8), and other language constructs, which can be automatically identified through data dependency analysis. The process of propagating taint labels involves not only propagation but also sanitization. In addition to the sanitization performed by directly assigning or copying safe (i.e., untainted) data to tainted memory region (e.g., `a = 1` on line 7), there are also sanitization situations related to semantics. For example, checking the content of `b` on line 10 excludes the possibility of overflow of `c = b + 1` on line 14. Due to the considerable engineering effort caused by the diversity of sanitization situations and the correlation of code semantics, previous work [12, 45, 74] ignores the setting of complex sanitization, which leads to considerable FPs, as discussed in [12, 49, 87].

Downloaded from the ACM Digital Library on April 7, 2025.

3.3 Problem 3: Formulating Inspection Rules for Various Sinks and Vulnerabilities

Taint analysis can discover dataflow sensitive vulnerabilities caused by external inputs, including various types of vulnerabilities listed in Table 1. These vulnerabilities can be triggered by different sinks in various ways, leading to a complex many-to-many relationship between vulnerability types and sinks. For each pair of a vulnerability type and a sink, it is often necessary to manually formulate a customized inspection rule. The process starts with identifying sinks, which can include not only standard C/C++ functions like `printf` and `system` but also third-party functions from various libraries like `BIO_printf` in OpenSSL [28]. When taint analysis is applied to a new binary that is linked with different libraries, human effort is indispensable to identify the sinks, because sink identification involves comprehending the behavior of functions in the binary. After determining sinks, inspection rules need to be formulated according to the potential vulnerability types.

For example, to inspect whether the `printf` call on line 9 in Figure 2 can trigger a CWE-134 vulnerability, we need to check whether the first argument (i.e., "%d") of `printf` is tainted; to inspect whether the call can trigger a CWE-190 vulnerability based on the first argument `%d`, we first need to check whether the second argument (i.e., `(ulong)e`) of `printf` is tainted; if yes, we need to further check whether the calculation process of this argument overflows. It is not scalable or practical to depend solely on human expertise and efforts to understand sink functions and vulnerability types, and manually formulate accurate and comprehensive inspection rules [12, 74, 88].

3.4 Advantages of LLMs in Vulnerability Inspection

The aforementioned specific implementation of the taint analysis often requires manual efforts combined together with human expertise of vulnerability analysis to establish taint rules. Comprehensive taint rules can reduce the workload of postprocessing analysis results, but also increase the complexity of the rules and the cost of analysis. For example, if the length of the tainted string with the null-terminator meets the requirements of the destination buffer of `strcpy`, the tainted string will not cause a buffer overflow vulnerability. Traditional taint analysis tracks the flow of taint labels from sources to sinks, so postprocessing to verify alerts is critical, such as bounds checking and value analysis.

Fortunately, LLMs inherently have the ability of human-like semantic understanding of code, unlike static or automated tools that rely on predefined patterns and rules. Certain software engineering tasks, such as code generation based on natural language descriptions [66, 96, 99] and code summarization [3, 82], have demonstrated the superb ability of LLMs to understand code.

To intuitively illustrate the benefits of LLM-assisted vulnerability inspection, we show the vulnerability analysis results produced by GPT-4.0 for the following three code snippets with this **Prompt Template (PT)**:

> *Use* `<function>` *as the taint source and mark the* `<parameter>` *parameter with the taint label for taint propagation. Determine whether the program has vulnerabilities. Pay attention to the buffer size and library function definition. If there is a vulnerability, please explain it according to the CWE type.*

where the pair (`<function>`, `<parameter>`) is replaced with (`fgets`, `first`) or (`recv`, `second`) accordingly.

Code Snippet without Vulnerabilities. The program in Figure 3 does not have any vulnerability and GPT-4 correctly produces the following analysis result:

> *In the provided code snippet, using* `fgets` *as the taint source and marking the first parameter with the taint label for taint propagation, the primary concern revolves around how data is handled and*

Downloaded from the ACM Digital Library on April 7, 2025.

```c
int main() {
    char a[50];
    char b[50];
    printf("Enter a string: ");
    fgets(a, sizeof(a), stdin);
    strcpy(b, a);
    return 0;
}
```

Fig. 3. A DB program that has no vulnerability.

```c
int main() {
    char a[60]; // The size of the buffer changes
    char b[50];
    printf("Enter a string: ");
    fgets(a, sizeof(a), stdin);
    strcpy(b, a); // A buffer overflow vulnerability here.
    return 0;
}
```

Fig. 4. A DB program, similar to Figure 3, uses `fgets` to receive user input (comments are added to aid understanding). It has a buffer overflow vulnerability. The reason for this program to have a buffer overflow vulnerability is that the buffer that `fgets` receives input is 60 bytes long, and `strcpy` may exceed the buffer `b` of the destination when copying.

> *copied between buffers. However, the specific use of* `fgets` *and* `strcpy` *in this context does not inherently introduce a classic buffer overflow vulnerability as might be seen with other functions like* `gets` *or unchecked uses of* `strcpy`*.*

Vulnerable Code Snippet with `fgets`*.* The program in Figure 4 is similar to Figure 3, but has a buffer overflow vulnerability. The following shows the correct analysis result by GPT-4:

> *In this updated code snippet, there is a clear vulnerability due to the use of* `strcpy` *to copy data from a larger buffer* `a` *(60 bytes) to a smaller buffer* `b` *(50 bytes) without checking the size of the input. This scenario introduces a classic buffer overflow vulnerability, which can be classified under CWE-120: Buffer Copy without Checking Size of Input (Classic Buffer Overflow).*

Vulnerable Code Snippet with `recv`*.* The program in Figure 5 is similar to Figure 3, but uses `recv` to receive user input. The following shows the correct analysis result by GPT-4:

> *In the provided code snippet, the* `recv` *function is used as the taint source, and the second parameter (*`a`*) is marked with the taint label for taint propagation. This setup introduces a potential vulnerability when the data received are copied from buffer* `a` *to buffer* `b` *using* `strcpy`*, without checking the size of the data being copied. This scenario can lead to a buffer overflow vulnerability, specifically classified under CWE-120: Buffer Copy without Checking Size of Input (Classic Buffer Overflow).*

Based on the three analysis results provided by GPT-4, it is evident that LLMs can understand and leverage deep semantic information about the code and the functions the code calls, to perform vulnerability checks. Therefore, LLMs have significant potential to automate certain aspects of taint analysis, a topic that will be further elaborated in the remainder of this article.

Downloaded from the ACM Digital Library on April 7, 2025.

```
1 int main() {
2     char a[50];
3     char b[50];
4     printf("Enter a string: ");
5     recv(SOCKET, a, sizeof(a), 0); // The taint source changes
6     strcpy(b, a); // A buffer overflow vulnerability here.
7     return 0;
8 }
```

Fig. 5. A vulnerable DB program that is the same as Figure 3 except using `recv` to receive user input instead of `fgets` (comments are added to aid understanding). The reason for this program to have a buffer overflow vulnerability is that `recv` does not guarantee a null-terminator, a subtle behavioral difference from `fgets`.

4 Overview of LATTE

We propose LATTE, a novel approach that leverages the code understanding capabilities of LLMs to assist in taint analysis for identifying potential vulnerabilities in binaries. This approach raises the key **Research Question (RQ)**: *How can we practically and effectively prepare prompts to guide LLMs in performing vulnerability inspection?* Effective prompts must provide LLMs with sufficient code context and clear analysis instructions. However, achieving this requires overcoming two main challenges.

Challenge 1: Large Code Size and Complexity vs. Limited Token Context of LLMs. The size and complexity of binary code make it impractical to directly feed an entire binary program to LLMs for analysis. LLMs have limits on the number of tokens and the amount of context they can process. When these limits are exceeded, LLMs may lose track of previously analyzed information, affecting subsequent code analysis. Even without these input limitations, effectively inspecting vulnerabilities across the entire codebase of a real-world software project remains a daunting challenge.

Solution to Challenge 1. Incorporating the principles of taint analysis, we extract DFs for LLMs to analyze. By slicing the program, we reduce the analysis complexity for LLMs. This method employs taint analysis principles to conduct dependency analysis on dangerous data related to the program's external input. However, unlike traditional taint analysis, we do not perform sanitization and vulnerability inspections; instead, we focus solely on extracting DFs. *A DF is defined as a slice of functions involved in data dependencies between the external data input function and potential destinations that could lead to vulnerabilities* (see Sections 5.1 and 5.2). Through this process, multiple DFs can be extracted from the program, forming a superset of potential vulnerabilities. This approach partitions the analysis space for LLMs, reducing the overhead of complex taint analysis algorithms and minimizing the risk of FNs. The extracted DFs provide LLMs with relevant and contextually concise input, enhancing the accuracy and efficiency of vulnerability analysis.

Challenge 2: Obscurity of Vulnerability Discovery vs. Generality of Instructions for LLMs. Vulnerability discovery inherently involves exploring the unknown. Instructing LLMs to perform similarity analysis based on known vulnerabilities limits the ability of LLMs to detect unknown vulnerabilities. Additionally, it is impractical to provide examples of all possible vulnerability patterns in the prompt and use few-shot learning for analysis [16]. The instructions for vulnerability checking described in the prompt should remain general, requiring no additional effort when the code or vulnerability types change.

Solution to Challenge 2. Inspired by the idea of manual code auditing that usually splits a vulnerability analysis task into multiple smaller sub-tasks, we instruct LLMs to analyze the DF step by step in a similar manner. When manually auditing code, we often use functions as units to gradually understand the entire program. In addition, manual audits often determine whether vulnerabilities

Downloaded from the ACM Digital Library on April 7, 2025.

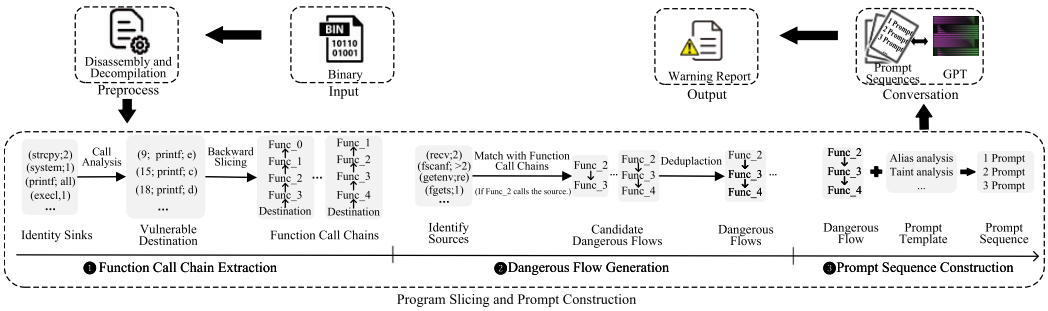


Fig. 6. Workflow of LATTE.

exist in a general way by understanding the definition of functions and the processing flow of data, rather than finding similarities through specific vulnerability examples. Therefore, taking the functions in the DF as units, we construct a comprehensive PS corresponding to the DF, which specifies the data source to carry out dataflow tracing and vulnerability inspection (see Section 5.3). This PS allows LLMs to complete the vulnerability analysis of the DF in the form of a conversation. Consequently, LLMs can explore and discover vulnerabilities more effectively, even in scenarios with different binaries or vulnerability types.

4.1 Workflow of LATTE

LATTE utilizes the idea of *code slicing* and *task splitting* to build prompts to instruct LLMs for vulnerability inspection. The overall workflow of LATTE is depicted in Figure 6, primarily comprising three modules. First, we *preprocess* the input binary using disassembly and decompilation techniques. This module provides the recovered code structures for further analysis. In the second module, we *slice* the DFs of the decompiled program, extracting relevant code snippets involving data dependencies between external inputs and VDs. For each DF, we generate the corresponding *PS* that instructs the LLM. Then, the third module uses the generated PS to engage in a *conversation* with the LLM to step-by-step guide the DF analysis for discovering vulnerabilities.

The code slicing and prompt construction module serve as the core of LATTE. We further explain the three phases of this module.

Phase 1: Function CC Extraction. In this phase, we begin by identifying VDs (i.e., call sites of sinks) within the binary. To achieve this, we utilize the LLM to analyze each external function to identify sinks and their related parameters, which may potentially trigger vulnerabilities. We consider the calls to these sinks as the VDs. Next, we conduct backward data dependency analysis, encompassing both intraprocedural and interprocedural aspects, starting from each VD. This analysis helps us reconstruct the function CCs leading to these VDs (details in Section 5.1). Function CCs are defined as below.

Definition 4.1 (CC). A CC is a sequence of functions $[f_1, f_2, \cdots, f_n]$, where f_n contains a VD and $\forall i \in [1, n) : f_i$ calls f_{i+1}.

Phase 2: DF Generation. The second phase focuses on identifying external input sources and matching them to the previously generated function CCs to generate DFs. To achieve this, we once again rely on the LLM to identify sources from external functions and determine their calls. Then, we match the caller of the sources within the function CCs. If the caller exists in a chain, we consider the chain as a **Candidate DF (CDF)** whose starting point is the caller. By applying a

Downloaded from the ACM Digital Library on April 7, 2025.

sequence-sensitive filtering process, we eliminate repeated flows among candidate CDFs, resulting in a set of distinct DFs (details in Section 5.2).

Definition 4.2 (DF). A DF is a CC = $[f_1, f_2, \cdots, f_n]$, where f_1 contains at least one call to a taint source, f_i $(1 < i \leq n)$ may contain calls to taint sources, f_n contains a VD, and the external data obtained from any of the taint sources called on the chain can possibly flow to the VD.

Phase 3: PS Construction. The third phase involves constructing PSs by combining the identified DFs with specific analysis tasks. For each DF, we create the corresponding PS using functions as units. To be applicable to various binaries, we design three PTs to complete the construction of prompts. The first PT is used to initiate a conversation, which combines with the first function in the flow to clarify the source of analysis, and instructs the LLM to perform data aliasing, data dependency, and taint analysis. We then use a second template combined with each function in the DF to recursively construct prompts for dataflow analysis until reaching the end of the DF. The last prompt instructs the LLM to initiate the final vulnerability analysis (details in Section 5.3).

5 LATTE Design

We introduce a novel technique for PS construction based on DF analysis, which enables LLMs to conduct effective vulnerability inspections in binaries. DFs are obtained by performing backward data dependency slicing from calls to sinks and matching sources. The analysis of DFs focuses solely on data dependency analysis and does not involve taint sanitation or vulnerability inspection. Based on the extracted DFs, combined with the task of program analysis, the LLM inspects DFs for potential vulnerabilities. Combined with Algorithm 1, we next explain the specific design of the program slicing and prompt construction process.

5.1 Function CC Extraction

In the function CC extraction phase, we focus on extracting function CCs that are relevant to potential vulnerabilities in the binary. This process involves two main steps: identification of VDs and backward slicing.

5.1.1 Identification of VDs. The first step is to find the call sites of sinks within the binary. To inspect all potential vulnerabilities, we need to locate all sinks present in the binary. Sinks often depend on external libraries linked to the binary, as mentioned in Section 3. To identify sinks, we leverage the power of the LLM. Specifically, we extract externally linked functions from the DB into a list named $\mathsf{F}_{external}$. For dynamically linked libraries, the names of external functions are preserved in the function table of the binary, so we can provide the function names to the LLM for analysis. For statically linked libraries, the body of the function is included in the binary. We utilize the LLM for code summary analysis of the external library function to identify sinks. As shown in Figure 7, we take dynamic linking as an example to illustrate this process. The form of static linking is just to replace the function name with the function body. The prompt first assigns the program analyst persona, because the role setting performs better in statistical results [93]. Then, the prompt clearly describes the purpose of the analysis, i.e., determine whether the function can be used as a sink in taint analysis. Moreover, we use the one-shot approach to give an example to standardize the output of LLMs. The output of LLMs is not necessarily completely accurate or stable. We can remove the "without additional explanation" indication in the prompt to use the description of the function in the response to assist in the judgment of sinks. For any functions f that is determined to be a sink, we store the pair $(f; \mathsf{Para})$ of the function name and parameter that needs to be checked, into a set named sinks on line 5 in Algorithm 1 for subsequent use. It is worth noting that the analysis results of (sink; Para) can be reused if another binary uses the same

Downloaded from the ACM Digital Library on April 7, 2025.

Algorithm 1: Program Slicing and Prompt Construction.

Input: DB: decompiled binary
Input: PTs: prompt templates
Output: PSs: prompt sequences for vulnerability inspection

```
// Function Call Chain Generation
1  F_external ← ExtractExternalFunc(DB)
2  sinks ← ∅
3  foreach f ∈ F_external do
4      if LLM thinks f is a taint sink with the parameter(s) Para causing the vulnerability then
5          sinks ← sinks ∪ {(f; Para)}
6  vulnerable destinations VDs ← ∅
7  foreach (sink; Para) ∈ sinks do
8      callsites ← find all the call sites of sink
9      foreach Loc ∈ callsites do
10         Arg ← find the actual argument for the parameter Para
11         VDs ← VDs ∪ {(Loc; sink; Arg)}
12 call chains CCs ← ∅
13 foreach VD in VDs do
14     CC_1, CC_2, ... ← BackwardSlicing (DB, VD)
15     CCs ← CCs ∪ {CC_1, CC_2, ...}
// Dangerous Flow Generation
16 sources ← ∅
17 foreach f ∈ F_external do
18     if LLM thinks f is a taint source with the parameter(s) Para storing external data then
19         sources ← sources ∪ {(f; Para)}
20 candidate dangerous flows CDFs ← ∅
21 dangerous flows DFs ← ∅
22 foreach CC ∈ CCs do
23     if the source of sources exists in CC and has data dependency with the VD then
24         CDF ← change the starting function to the caller of source
25         CDFs ← CDFs ∪ CDF
26 DFs ← Deduplicate (CDFs)
// Prompt Sequence Construction
27 prompt sequences PSs ← ∅
28 foreach DF ∈ DFs do
29     PS ← ConstructPromptSeq (PTs, DF)
30     PSs ← PSs ∪ PS
31 return PSs
```

dynamic library. This analysis is only necessary when the binary uses a different dynamic library that has not been analyzed before.

The output result is a set of pairs `(sink; Para)`, as shown on line 5 in Algorithm 1. For example, `(printf; 1), (printf; 2),` $\cdots$ indicate that every parameter of the `printf` function needs to be checked for potential vulnerabilities. After identifying the sinks and the corresponding Para, we traverse the binary to locate the call sites of each sink, forming the VD list of the triplet

Downloaded from the ACM Digital Library on April 7, 2025.

Model Input

As a program analyst, is it possible to use a call `printf` as a sink when performing taint analysis? If so which parameters need to be checked for taint. Please answer yes or no without additional explanation. If yes, please indicate the corresponding parameters. For example, the `system` function can be used as a sink, and the first parameter needs to be checked as `(system; 1)`.

Model Output

Yes, `printf` can be used as a sink. The parameters to be checked for taint are:
Format string `(printf; 1)`
Any additional parameters being formatted `(printf; 2, 3, ...)`

Fig. 7. Use GPT-4.0 to identify sinks.

`(Loc; sink; Arg)`. For example, the first parameter in the `printf` calls in Figure 2 is a constant and does not need to be checked, so Figure 2 contains three VDs: `(9; printf; e)`, `(15; printf; c)`, and `(18; printf; d)`.

5.1.2 Backward Slicing. The second step, backward slicing, involves obtaining the operations performed on the VD's arguments before the call to the sink. This step relies on both intraprocedural and interprocedural dataflow analyses. We adopt a depth-first backward approach to traverse the call graph and analyze dataflows backward from each VD = (Loc; sink; Arg) to generate CCs, as shown on lines 12–15 in Algorithm 1.

LATTE relies on Ghidra [62] to perform call graph analysis. Specifically, we obtain the address of each function[2] by traversing the functions identified by Ghidra. Then, `call` and `jump` instructions are found in each function, and the memory addresses used in these instructions are matched with the function addresses to identify the relationship between callers and callees and further build the call graph. Based on the call graph, we first perform flow-insensitive and path-insensitive intraprocedural data dependency analysis within the function f where VD is located. For loops within f, if duplicate paths are found, the loop is exited. In addition, we perform pointer alias analysis with the help of variable description definitions and addressing methods [11]. If direct data dependencies are detected between the VD's vulnerability trigger-related arguments (i.e., `arg`) and the f's parameters, we extend our analysis to an interprocedural level. The calling context is considered by recording the function names of different callers to form multiple CCs. Therefore, we recursively analyze the dependencies until there are no dependencies on caller parameters or no caller. To manage the complexity of recursive calls and path explosion, we limit the context call depth to 50. Finally, we add the CCs corresponding to different VDs to the set CCs.

We use an example to demonstrate the above analysis process. CCs, as shown in Figure 8, show a partial call graph, where the library function `system` is the sink. The VD = `(10; system; a)` is in `fun1`, and we initialize its CC = `[fun1]`. There is a data dependency between `a` and `para2` of `fun1`. So we continue to analyze the call sites of fun1 and update CCs to {`[fun2, fun1]`, `[fun3, fun1]`}. In `fun3`, the argument corresponding to the parameter `para2` of `fun1` is b, and b has a data dependency with `para2` of `fun3`. Therefore, we continue to analyze the callers of `fun3` until there is no caller or no dependency on the parameters.

[2]A function address refers to the memory location where the code of the function begins in memory.

Downloaded from the ACM Digital Library on April 7, 2025.

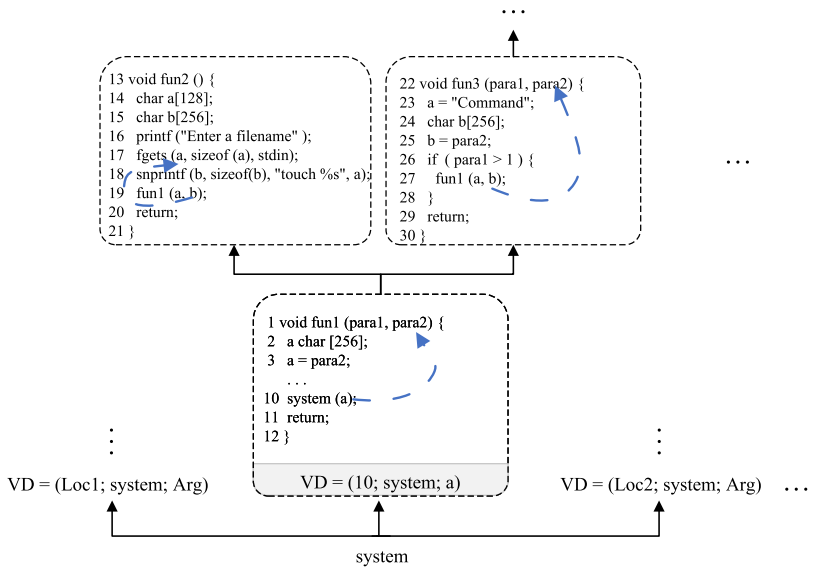


Fig. 8. An example call graph of a program, where each node is a function definition. Black arrows represent function call relationships. Blue arrows represent data dependency relationships. The figure directly shows the two CCs of *VD* = (`10`; `system`; `a`), namely, {[`fun2`, `fun1`], [· · · , `fun3`, `fun1`]}.

5.2 DF Generation

In the DF generation phase, our goal is to identify CCs in which external input can reach a VD. This process involves two main steps: We first need to identify the external input source in the program and then perform matching and deduplication in CCs to determine DFs.

5.2.1 Source Identification. The first step is to identify sources in the binary. Similar to the identification of sinks in Section 5.1.1, we also utilize the LLM for this task. In this case, the analysis task is changed to find functions that can directly receive external inputs. We extract external link functions in the binary and provide the function name or function body so that LLM can perform auxiliary analysis to determine whether the function can be used as a taint source. For any function f that is determined to be a source, we store the pair (f; Para) of the function name and parameter that needs to be checked into a set named sources on line 5 in Algorithm 1 for subsequent use. For instance, Figure 9 indicates that the `fscanf` function is used as the source, and the second parameter stores the external input data. Obviously, this response is not entirely accurate. External input should be stored in the corresponding pointer parameter after the format string. Therefore, we need to make corrections based on slight expert experience. We use (`fscanf; > 2`) to indicate that we need to analyze the parameters after the second parameter of the `fscanf` function. Like the identification of sinks, the source identification results can also be reused for different binaries that uses the same dynamic library.

5.2.2 Matching and Deduplication. With the identified set of sources {(source; Para)}, we proceed to match them in the extracted CCs to identify DFs. Specifically, for each CC, we check whether the function f of the CC call the identified sources. If so, we further analyze whether the argument corresponding to Para of the source overlaps with the dataflow of the VD. If there is overlap, the functions from f to the end in the CC are considered a CDF. As a CC may generate

Downloaded from the ACM Digital Library on April 7, 2025.

Model Input

As a program analyst, is it possible to use a call to `fscanf` as a starting point (source) for taint analysis? If the function can be used as a taint source, which parameter in the call stores the external input data. Please answer yes or no without additional explanation. If yes, please indicate the corresponding parameters. For example, the recv function call can be used as a taint source, and the second parameter as a buffer stores the input data as `(recv; 2)`.

Model Output

Yes, the `fscanf` function call can be used as a taint source, and the second parameter (format string) stores the input data `(fscanf; 2)`.

Fig. 9. Use GPT-4.0 to identify sources.

multiple CDFs due to multiple sources, we only keep the longest CDF within the same CC. Additionally, different CCs may produce the same CDF or subchain. To avoid redundancy and reduce unnecessary inspection requests, we filter out CDFs that are subchains of other CDFs. Finally, we obtain a set of DFs for the next step to construct PSs.

We use CCs = {[fun2, fun1], [$\cdots$, fun3, fun1]} in Figure 8 to illustrate the above process. In fun2, the taint source fgets is called and the first argument a has a data dependency with the argument b of func1 (the snprintf in line 18 of Figure 8 stores a in b). Therefore, [fun2, fun1] is a CDF. We assume that CC = [$\cdots$, fun3, fun1] forms {[fun5, fun4, fun3, fun1], [fun6, fun4, fun3, fun1]} after the recursive analysis. If the taint source is called in fun4 and fun6, the corresponding CDFs is {[fun4, fun3, fun1], [fun6, fun4, fun3, fun1]}. But [fun4, fun3, fun1] is a subchain of [fun6, fun4, fun3, fun1], so the former is discarded. The final DFs are {[fun2, fun1], [fun6, fun4, fun2, fun1]}.

5.3 PS Construction

In the PS construction phase, our objective is to create PSs that instruct the LLM to perform vulnerability inspection based on identified DFs. To ensure that the PS effectively instructs the LLM and remains valid and effective for different, diverse programs, we propose PTs that are applicable to all DFs and can be instantiated to fit each DF. We follow a rigorous iteration and optimization process that ranged from initial prompt design to testing and feedback analysis. The final PTs include the following key elements for effective analysis:

—Clearly state the analysis tasks. Precise instructions for vulnerability analysis are essential to prevent irrelevant content.
—Enrich the context of tasks. Additional instructions related to vulnerability analysis, such as alias analysis and focusing on tainted dataflows, enhance accuracy.
—Persona assignment. Assigning appropriate roles improves the professionalism of LLMs in responding to descriptions.
—Splitting of analysis tasks or data. For large data to be analyzed, splitting or preprocessing the data helps improve accuracy.
—The specified output format. The determined output format facilitates analysis for downstream tasks.

Downloaded from the ACM Digital Library on April 7, 2025.

Template for Start Prompt

As a program analyst, I give you snippets of C code generated by decompilation, using <function> as the taint source, and the <parameter> parameter marked as the taint label to extract the taint data flow. Pay attention to the data alias and tainted data operations. Output in the form of data flows.
<Code to be analyzed>

(a) Template for the start prompt.

Template for Middle Prompt

Continue to analyze function according to the above taint analysis results. Pay attention to the data alias and tainted data operations. (Note the new taint source <function> and the <parameter> parameter marked as the taint label.)
<Code to be analyzed>

(b) Template for the middle prompt.

Template for End Prompt

Based on the above taint analysis results, analyze whether the code has vulnerabilities. If there is a vulnerability, please explain what kind of vulnerability according to CWE.

(c) Template for the end prompt.

Fig. 10. The PTs for constructing the PS that instruct the LLM to inspect vulnerabilities.

According to these elements, we design three templates shown in Figure 10. The PTs are inspired by the general approach of manual code auditing. Specifically, data dependency analysis starts from the function containing the external data source and progresses through the function CC until the last function in the DF. If a DF contains only one function, then the PS for it includes a start prompt and an end prompt. For a DF consisting of more than one function, the middle template is used recursively to complete the PS construction. This structured approach ensures that LATTE performs vulnerability analysis effectively and consistently across different scenarios. The detailed explanation of PTs design is as follows.

Start Template (Figure 10(a)). Initializes the analysis by specifying the task and context clearly. The start prompt assigns the persona of the LLM and explains the analysis background and tasks. Specifically, we instruct the LLM to conduct the following conversion in the role of program analyst. Then, we clarify the taint source of program analysis. For example, for the DF = [fun2, fun1] shown in Figure 8, the pair (<function>, <parameter>) in Figure 10(a) is replaced with (fgets, first). At the same time, the body of first function fun2 is appended at the end of the prompt for analysis. In the description of the analysis task, we mainly focus on the propagation of tainted data, so we emphasize the analysis of data dependencies. To conduct more accurate analysis, we also emphasize the focus on data aliases and tainted data operations.

Middle Template (Figure 10(b)). Recursively used to handle intermediate steps in the DF, enriching the context and focusing on relevant dataflows. Specifically, the middle prompts are reused to describe functions in a DF other than the first function. Thanks to the short-term memory capability

Downloaded from the ACM Digital Library on April 7, 2025.

of the LLM itself, we can complete data dependency analysis between functions. Therefore, the middle prompt needs to indicate that the taint analysis is continued based on the analysis results of the previous function. Similarly, we put the function body to be analyzed at the end of the prompt. For example, for the DF = [fun2, fun1] shown in Figure 8, the body of function fun1 is appended at the end of the prompt for analysis. Moreover, if there is a source in the current function, we fill in the source name and the corresponding parameter at <function> and <parameter> as in the start template. Otherwise, taint propagation analysis is performed based on the analysis results of the previous function and the prompt does not include the content in parentheses (i.e., "(Note the new taint source ... as the taint label." in Figure 10(b)).

End Template (Figure 10(c)). Concludes the analysis, ensuring that all necessary information is captured and formatted correctly. Specifically the end template is used after all functions in have been analyzed. The main task of the end template is to perform vulnerability inspection based on the analysis results of all DF functions. At the same time, to facilitate vulnerability identification and analysis, we also let the end template classify vulnerabilities based on CWE. It is worth mentioning that this not only provides a more detailed vulnerability analysis description, but also provides patching suggestions based on the code context.

Assuming that the DF only contains the foo function in Figure 2, the running example of LATTE instructing the LLM to perform vulnerability analysis is shown in Figure 11. First, the output of LLM clearly describes the taint propagation process. The model then gives the location and reason why the vulnerability was triggered in the output of the vulnerability analysis. Therefore, we can not only see the vulnerability check results but also further verify the vulnerability or patch the vulnerability based on the analysis process in the response of LLMs.

6 Evaluation

The evaluations of LATTE are designed to address the following five RQs:

RQ1: How well does LATTE inspect vulnerabilities in binaries? Explain from the following three perspectives:

(1) How effective is LATTE in identifying security vulnerabilities?
(2) How stable is LATTE in vulnerability inspection result?
(3) How efficient is LATTE in terms of processing time and monetary cost?

RQ2: How effective is the identification of security sensitive functions (sinks) and external input sources (sources)?

RQ3: How effective is the extraction of DFs?

RQ4: What is the performance impact of PS on LATTE?

RQ5: Does LATTE scale to real-world binary vulnerability analysis?

6.1 Evaluation Setup

Implementation. The implementation of LATTE is based on the reverse engineering framework Ghidra [62, 76] and the LLM GPT-4.0 [63]. Below, we outline the overall implementation process. The first step involves loading the target binary into Ghidra, where the custom Ghidra plugin is utilized to automate extraction tasks. Through the plugin, LATTE retrieves critical information from the binary, such as decompiled code, the call graph, control flow graph, import table, and additional metadata necessary for in-depth analysis. Once the binary data are collected, LATTE conducts detailed program analysis to extract DFs that are relevant to vulnerability triggering. Using predefined PTs, LATTE constructs PSs that correspond to each DF. Each PS is specifically designed to interact with GPT-4.0, setting up a session in which the LLM conducts a vulnerability assessment

Downloaded from the ACM Digital Library on April 7, 2025.

Model Input

As a program analyst, I give you snippets of C code generated by decompilation, using fscanf as the taint source, and third parameter marked as the taint label to extract the taint data flow. Pay attention to the data alias and tainted data operations. Output in the form of data flows.

```
void foo(void) {
    char a, b, c, d;
    ...
```

Model Output

To analyze the data flow in the provided C code, I'll focus on the taint source, which is the fscanf function using the third parameter (&a) as the taint label. I'll also take into account the data aliasing and tainted data operations. Here's a step-by-step analysis of the data flow:
1: fscanf(stdin, "%d", &a); - a is tainted by the input from stdin.
2: b = a; - b gets tainted with the same value as a.
3: c = 0; - c is not tainted.
4: a = 1; - a is no longer tainted, overwritten with a constant value.
5: e = b + 1; - e is tainted by the value of b, which is tainted.
6: printf("%d",(ulong)e); - e is printed, but it's cast to ulong.
7: if (b == 127) - Condition checks tainted value of b.
If true:
...

Model Input

Based on the above taint analysis results, analyze whether the code has CWE-190 integer overflow vulnerabilities.

Model Output

The code snippet you provided does indeed contain a potential integer overflow vulnerability, specifically related to CWE-190 (Integer Overflow or Wraparound).
Let's examine the vulnerable part of the code: d = c + 1;
Here, c is potentially tainted by the input value of b. If b is close to the maximum value that a char can hold (which is typically 127 for signed char), then adding 1 to it (c + 1) can result in an overflow condition, leading to undefined behavior.
Although c is assigned the value of b + 1 in an earlier part of the code (c = b + 1;), the subsequent addition of 1 to c (d = c + 1;) is where the overflow may occur.
...

Fig. 11. Based on the *foo* function in Figure 2, the process of using GPT-4.0 to analyze and check bugs is shown. The final model output correctly identifies that the $d = c + 1$ operation of the *foo* has a potential integer overflow vulnerability.

of the associated DF. As GPT-4.0 processes each PS, it evaluates the corresponding DF for potential vulnerability issues. Once the analysis across all DFs within a binary is completed, LATTE saves the results generated by GPT-4.0. These results, stored as individual files, are organized for further verification and validation, supporting a comprehensive review of any identified vulnerabilities.

Benchmarks. To thoroughly and fairly evaluate the capabilities of LATTE, we used the Juliet Test Suite [23] and Karonte dataset [74]. We first selected the benchmarks included in the standard dataset Juliet Test Suite (v1.3) according to Table 1 to evaluate RQ1–RQ4 (Juliet has no benchmarks for CWE-120 and we used the Karonte dataset for CWE-120). To ensure that our evaluation

Downloaded from the ACM Digital Library on April 7, 2025.

Table 2. Evaluation Results of Vulnerability Inspection Based on Juliet

	CWE-78 (960/960)[a]			CWE-134 (1,200/1,200)[a]			CWE-190 (2,860/2,860)[a]			CWE-606 (240/240)[a]		
	Emtaint	Arbiter	LATTE	Emtaint	Arbiter	LATTE	Emtaint	Arbiter[b]	LATTE	Emtaint	Arbiter	LATTE
TP	820	408	892	1,115	1,166	1,151	×	2,091	1,773	×	×	210
FN	140	552	68	85	34	49	×	769	1,087	×	×	30
TN	960	430	960	1,132	30	1,102	×	444	1,779	×	×	142
FP	0	530	0	68	1,170	98	×	2,416	1,081	×	×	98
Accuracy	92.7%	43.64%	96.46%	93.63%	49.83%	93.88%	×	44.32%	62.1%	×	×	73.33%
Precision	100%	43.5%	100%	94.25%	49.91%	92.15%	×	46.39%	61.12%	×	×	65.26%
Recall	85.42%	42.5%	92.92%	92.92%	97.17%	95.92%	×	74.68%	68.19%	×	×	87.5%
F1 score	92.14%	43.25%	96.33%	93.58%	65.95%	93.99%	×	56.76%	62.05%	×	×	74.24%
Avg. time (seconds)	5.2	122	13.5	5.6	73.6	14.2	×	7.8	15.1	×	×	13.9

× indicates that the vulnerability type is not supported.

[a] *Number of test subjects containing vulnerabilities/Number of test subjects not containing vulnerabilities.*

[b] Arbiter relies on symbol table, such as `printIntLine`, `printHexCharLine`, and `printLongLongLine`, to assist in overflow judgment. If LATTE is also analyzed based on the binary with the symbol table, TP is 2,576, FN is 284, TN is 1,939, FP is 921, Precision is 73.66%, Accuracy is 78.93%, Recall is 92%, F1 is 81.04%.

mirrors real-world scenarios, we removed debug information and symbol tables from the compiled benchmark programs. Furthermore, since LATTE aims to detect vulnerabilities caused by external inputs, test subjects with internal data as a source have been removed. The number of test subjects used for evaluation is listed in Table 2. Second, we performed RQ5 evaluation on real firmware dataset provided by Karonte to illustrate LATTE's ability to find vulnerabilities in the real world. Since the source code of firmware in Karonte is not publicly accessible, and thus not in the training sets of LLMs. The evaluation on Karonte helps to investigate whether LATTE suffers from the data leakage problem [94].

Baselines. We selected Emtaint [12] and Arbiter [88] for comparison on the Juliet Test Suite. These techniques represent the state of the art as shown in Table 1. For the Karonte dataset, we used Karonte [74] as another baseline for comparison, because Karonte is designed specifically for analyzing firmware. We did not include CodeQL [27], QueryX [33], or Joern [29] as baselines due to their reliance on customized queries for bug detection [33].

Metrics. In the evaluation based on the Juliet Test Suite, we measured the effectiveness of LATTE using six indicators: **True Positive (TP)**, FN, **True Negative (TN)**, FP, accuracy ($\frac{TP+TN}{TP+TN+FP+FN}$), and F1 score ($\frac{2*precision*recall}{precision+recall}$, where $precision = \frac{TP}{TP+FP}$, $recall = \frac{TP}{TP+FN}$). For the evaluation on the real-world firmware dataset provided by Karonte, since there are no labels, we mainly measured the TP and FP in the analysis alerts.

Running Environment. All experiments were performed on a Linux workstation with an Intel Core i7-8750H CPU and 64 G RAM.

6.2 Vulnerability Inspection (RQ1)

6.2.1 Effectiveness. LATTE performs vulnerability inspection on stripped binaries compiled from Juliet. The name of the test subject describes the CWE vulnerability types it contains. However, there are overlaps between CWE vulnerability types (Section 2.1). To facilitate the verification of the results, we only determine whether the CWE vulnerability type specified by the test subject exists. In evaluating LATTE, we performed five rounds of testing to ensure robustness and accuracy. As shown in Table 2, the inspection accuracies of the four types of vulnerabilities CWE-78, CWE-134, CWE-190, and CWE-606 are 96.46%, 93.88%, 62.1%, and 73.33%, respectively, and the average accuracy is 77%.

LATTE outperforms both Emtaint and Arbiter, in terms of accuracy and F1 score. Note that in the case of CWE-190 (Integer Overflow or Wraparound), Arbiter can detect 318 more TPs than LATTE.

Downloaded from the ACM Digital Library on April 7, 2025.

Table 3. Comparison of Vulnerability Checking Accuracy of LATTE Based on GPT-4.0 and GPT-3.5

	GPT-4.0	GPT-3.5
CWE-78	96.46%	78.2%
CWE-134	93.88%	75.9%
CWE-190	62.1%	54.5%
CWE-606	73.33%	61.2%
Average	81.44%	67.45%

However, this is attributed to the fact that Arbiter relies on the symbol table for inspection, and if this information is provided to LATTE, LATTE can outperform Arbiter by detecting 485 more TPs.

Analysis of FNs and FPs. We further analyzed the reasons behind FNs and FPs in LATTE vulnerability checks. The first reason is that the DFs of test subjects are not always correctly extracted. This may occur due to program analysis problems, such as indirect calls and pointer aliases, which can lead to false or failed extraction of DFs (details in Section 6.4). The second reason is related to the creativity of GPT-4.0 which can lead to different responses to the same PS, resulting in unstable analysis results. However, the vulnerability inspection task itself has the property of exploration rather than simple rule matching. Therefore, the instability brought about by the creativity of GPT-4.0 is not necessarily bad for vulnerability inspection. We further analyzed the stability of the LATTE result (details in Section 6.2.2).

GPT-4.0 vs. GPT-3.5. LATTE based on GPT-4.0 can perform effective taint-style vulnerability inspection. We further used GPT-3.5 to conduct an ablation experiment on the model. Table 3 shows that LATTE based on GPT-4.0 performs better than GPT-3.5 on each vulnerability type. The average accuracy of LATTE based on GPT-4.0 is better than that of GPT-3.5 13.99%. This illustrates that the ability of the model itself directly affects the analysis effect of LATTE. With the development of LLMs, the effectiveness of vulnerability checks will also increase. The design of LATTE allows easy switching between different LLMs.

6.2.2 Stability. As an artificial intelligence generation model, GPT-4.0 exhibits a certain degree of randomness in the generated response. The temperature parameter plays a significant role in controlling the creativity and diversity of the content produced by GPT-4.0. The temperature value is a floating-point number between `0` and 1, where higher values result in more diverse and random outputs, while lower values produce more conservative and deterministic outputs. We conducted two sets of experiments to evaluate the stability of LATTE analysis results and the effect of temperature on the results.

Analysis Stability at Temperature 0.5. To strike a balance between creativity and stability, we used a median temperature value of `0.5` in the five rounds of testing in Section 6.2.1. We measured the stability by evaluating the intersection of TPs among the five rounds. Figure 12 shows small fluctuations in the number of TPs in different rounds of the same CWE type. From left to right in Figure 12, the recall rates of five-round intersection are 90.42%, 88.42%, 58.04%, and 82.5%, respectively (the average recall rate is 79.85%). This indicates that the stability of LATTE is acceptable. However, we noticed relatively large fluctuations in the CWE-190. The reason for the fluctuation of CWE-190 is consistent with the problem mentioned above (Section 6.2.1), which is caused by incorrect data type judgments without the assistance of function names.

Downloaded from the ACM Digital Library on April 7, 2025.

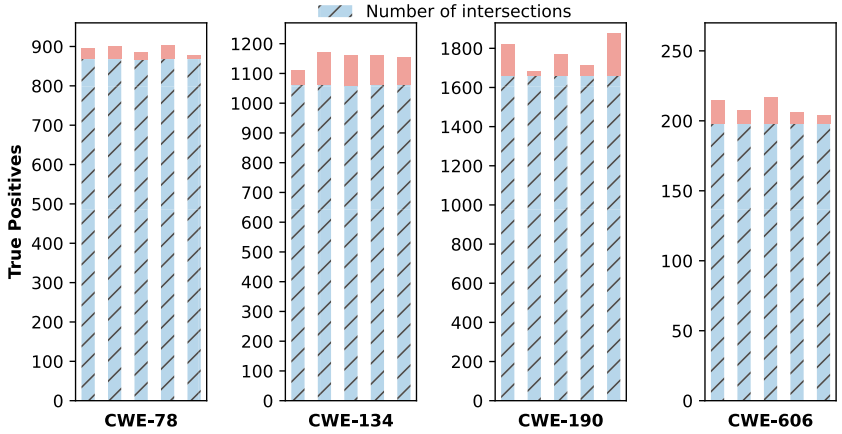


Fig. 12. The TP intersection among the five test rounds indicates the stability of LATTE. The intersection numbers of the four vulnerability types are 868, 1,061, 1,660, and 198.

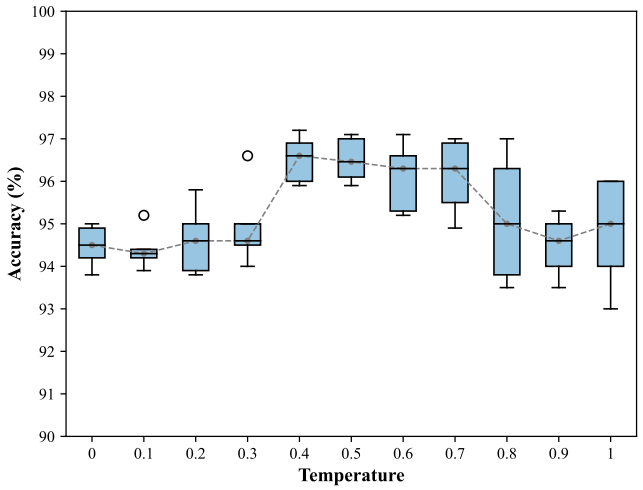


Fig. 13. Accuracy of CWE-78 analysis for LATTE at different temperatures.

Impact of Temperature on Analytical Results. To explore the impact of temperature on LATTE, we conducted 11 experiments with different temperature values. Each experiment was performed five rounds to ensure reliability. Figure 13 shows the accuracy results of CWE-78, where the trend generally follows a normal distribution, including the median, minimum, and maximum values. Moreover, we observe that when the temperature is less than `0.4`, the inspection results have a smaller distribution range. Conversely, when the temperature is greater than `0.6`, the inspection results are more scattered. The temperature of `0.4`–`0.6` can achieve a balance between stability and inspection effect. Similar trends were observed for the analysis of CWE-134, CWE-190, and CWE-606.

6.2.3 Efficiency. We analyzed the time overhead and the monetary cost of LATTE when inspecting for vulnerabilities. This process refers to the overhead of binary loading, DF extraction,

Downloaded from the ACM Digital Library on April 7, 2025.

and conversation with GPT-4.0 for vulnerability inspecting. Source and sink results can be reused between different binaries so they are preconfigured (evaluated in Section 6.3).

The average time taken to analyze different test subjects is summarized in Table 2. The results indicate that LATTE can complete the analysis of a test subject within 20 seconds. Preprocessing the test subject and extracting the DFs takes about 2 seconds, which is a relatively efficient process. The majority of the analysis time is spent on the conversation with GPT-4.0, as this step involves instructing the model and waiting for its response. The time taken for this step can be influenced by network quality and the response speed of GPT-4.0. Comparing LATTE with Emtaint and Arbiter, we find that the time overhead of LATTE is slightly greater than that of Emtaint, with a difference of about 8 seconds. This difference is considered acceptable given the automation that comes with using GPT-4.0.

Regarding the monetary cost, the evaluated cost of LATTE analyzing a DF is $0.35, which is considered acceptable compared to the expenditure of hiring a program analysis expert for manual inspection. Moreover, with the development of LLM technology, the cost of using such models is expected to decrease over time, while the performance is likely to improve, making it an even more attractive option for vulnerability analysis.

> *Answer to RQ1*: The average accuracy of LATTE vulnerability inspection is 77%. Compared to the state-of-the-art techniques Emtaint and Arbiter, LATTE outperforms them in different vulnerability types. Moreover, the average recall of the TP intersection across different test rounds is 79.85%, which shows that the inspection results of LATTE are stable.

6.3 Sink and Source Identification (RQ2)

LATTE reasonably identified sinks and sources as necessary for DF extraction. As shown in Table 4, we verified the results of sink and source identified by LATTE. For each test subject, the vulnerability is triggered by a specific pair of sink and source, such as `system` and `recv`. The expected pair of the test subject is indicated by its name. LATTE accurately covers both the sink and source of the target scenario in the test subject with 100%. The time overhead for identifying sinks and sources is proportional to the number of library functions. For the test subject in Juliet, 48 queries were required, with each query taking approximately 1 second. Furthermore, these results can be cached for all test subjects from Juliet without the need for further queries.

There are other functions in a test subject that can be used as sinks and sources, but they do not cause a vulnerability in the current test subject. The identified functions can be used as sinks and sources with average precision up to 82.1%. FPs are mainly attributed to the instability of GPT-4.0. For example, sometimes `recv` is mistakenly treated as an sink. This highlights the challenge of managing the creativity and diversity of the LLM, which can sometimes result in unintended outcomes. To alleviate this situation, we can analyze function semantics to filter FPs and utilize TPs to form sink and source collections to assist analysis. When evaluating real programs in Section 6.6, we use the correct sink and source results identified from the test subjects to analysis.

Compared with the state-of-the-art binary taint analysis techniques, such as Emtaint and Arbiter, LATTE improves the automation of taint analysis. Previous techniques primarily relied on manual analysis of function semantics to identify sinks and sources, making it difficult to compare identification results. However, the ability of LLMs to understand code and generate function summaries can be used to assist manual analysis and reduce manual effort. LATTE simplifies the analysis process and improve the efficiency of existing techniques by providing analysts with more program information.

Downloaded from the ACM Digital Library on April 7, 2025.

Table 4. Sink and Source Identification Results

	Avg. # Sink	Precision	Coverage[a]	Avg. # Source	Precision	Coverage[a]
CWE-78	6.12	85.42%	100%	4.72	76.17%	100%
CWE-134	6.71	86.2%	100%	4.52	80.21%	100%
CWE-190	5.2	90.21%	100%	5.63	72.14%	100%
CWE-606	6.79	85.18%	100%	4.46	81.11%	100%

[a]The proportion of identification results covering the test subject target scenario.

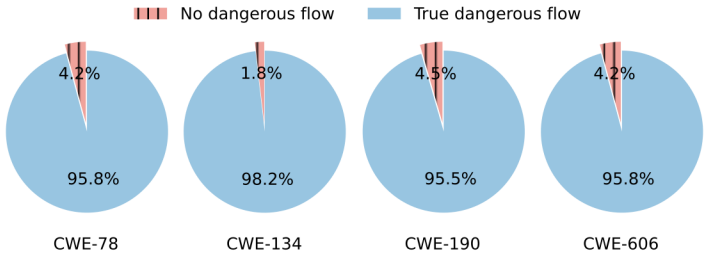


Fig. 14. DF extraction results.

Answer to RQ2: LATTE automatically identifies the sink and the source with an average precision of 82.1%. The identification results correctlycover the expected sink and source pairsof the test subjects 100%.

6.4 DF Extraction (RQ3)

The effectiveness of DF extraction affects the accuracy of GPT-4.0 vulnerability checking. The FN of DFs directly leads to the FN of LATTE vulnerability checks. To evaluate the effectiveness of DF extraction, we analyzed vulnerable test subjects (labeled as "bad") in Juliet. Vulnerable test subjects contain a potential vulnerability. We check whether the DF extraction is successful and whether the vulnerability triggering path between source and sink can be covered.

Figure 14 shows that among different types of vulnerable test subjects, LATTE successfully extracted the DF for more than 95% of the vulnerable test subjects. Moreover, the DF of each successfully extracted test subject is correct, i.e., covering the code snippets between source and sink that causes the test subject to trigger the vulnerability. The detailed numbers of DFs corresponding to each CWE type are 920, 1,178, 2,731, and 230, respectively. Each successfully extracted test subject corresponds to a unique DF. This indicates that LATTE's DF extraction is effective, as LATTE correctly captures potential paths leading to vulnerabilities in the most of vulnerable test subjects.

However, there are test subjects where LATTE did not successfully extract DFs (FNs). We investigated these test subjects and identified two primary reasons: ① Ghidra decompilation error. The decompilation engine may have problems parsing function call arguments, such as three arguments being mistaken for two. This leads to the breakage of taint data dependencies in the callee function, ultimately affecting the extraction of DFs. ② Pure virtual function calls. Pure virtual functions have no function body and must be implemented in derived classes. Test subjects with pure virtual function calls prevent LATTE from extracting the DF because there is no concrete implementation to analyze.

Downloaded from the ACM Digital Library on April 7, 2025.

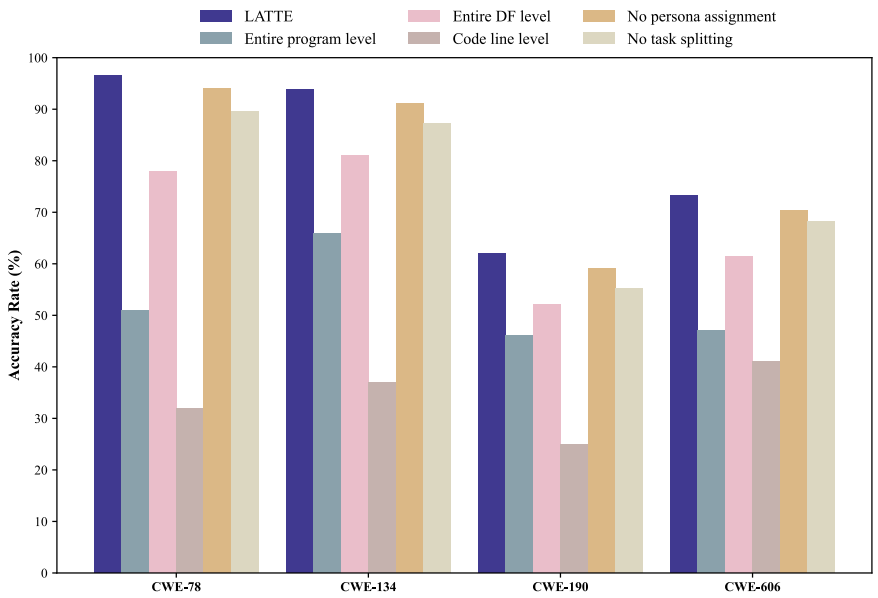


Fig. 15. Comparison of the impact of PS construction on vulnerability analysis.

Answer to RQ3: LATTE successfully and correctly extracts DFs covering more than 95 of test subjects containing potential vulnerabilities. This demonstrates the ability of LATTE to effectively extract DFs.

6.5 PS Construction (RQ4)

To evaluate the effectiveness of PS construction, we designed five sets of comparative experiments based on the factors that affect prompt construction: code slicing granularity and analysis task description granularity. Specifically, keeping the same analysis task description as LATTE, we perform prompt construction on ① the entire program and ② the entire DF, respectively. Moreover, we construct PSs based on more fine-grained slicing, that is, ③ only retaining the code lines related to the data dependency of the sink trigger argument. For analysis task description, we use functions in DF as units to construct PS consistent with LATTE, but we ④ cancel the persona assignment (i.e., the first prompt in PS does not indicate that the conversation will be conducted in the role of a program analyst) and ⑤ do not split tasks (i.e., directly instruct the GPT-4.0 to perform vulnerability analysis without mentioning subtasks such as taint analysis and data dependency analysis).

Based on the above principles, we construct corresponding prompts (or PSs) for vulnerability analysis on the test subjects in Table 2. As shown in Figure 15, LATTE outperforms the other five methods. Moreover, the p-values for the comparison pairs calculated using the Mann-Whitney U Test [83] shown in Table 5 are all less than `0.05`, indicating that the comparison results are statistically significant. Specifically, LATTE outperforms the approach on larger-grained slices (the entire program and the entire DF) and smaller-grained slices (code lines involved in tainted data) by an average accuracy of 28.7%, 13.3%, and 46.4%, respectively. In large-granularity slices, the entire DF is also better than the entire program with an average accuracy of 15.4%, which shows that directly providing a large number of code snippets at one time leads to a decrease in analysis accuracy. For smaller-grained slicing, GPT-4.0 produced responses that vulnerability

Downloaded from the ACM Digital Library on April 7, 2025.

Table 5. The p-Values Calculated by Mann-Whitney U Test to Illustrate the Statistical Significance of LATTE and Comparative Sets

	Entire Program	Entire DF	Code Line	No Persona Assignment	No Task Splitting
CWE-78	0.008	0.008	0.008	0.008	0.012
CWE-134	0.008	0.008	0.008	0.012	0.012
CWE-190	0.008	0.008	0.008	0.008	0.012
CWE-606	0.008	0.008	0.008	0.012	0.012

Table 6. Vulnerability Inspection Results Based on Real-World Firmware Dataset Collected by Karonte

Vendor	Samples	Karonte					Emtaint					Arbiter					LATTE				
		Alerts	TP	FP	Unique Bugs	Avg. Time	Alerts	TP	FP	Unique Bugs	Avg. Time	Alerts	TP	FP	Unique Bugs	Avg. Time	Alerts	TP	FP	Unique Bugs	Avg. Time
NETGEAR	17	36	23	13	19	17:13	849	849	0	47	00:05	131	63	68	26	02:20	142	124	18	54	00:38
D-Link	9	24	15	9	10	14:09	299	234	65	29	00:02	99	48	51	14	03:18	65	59	6	42	00:21
TP-Link	16	2	2	0	2	01:30	73	73	0	3	00:05	21	18	3	6	02:40	29	23	6	10	00:33
Tenda	7	12	6	6	6	01:01	362	362	0	19	00:05	49	22	27	6	03:25	19	16	3	13	00:29
Total	49	74	46	28	37	451:06	1,583	1,518	65	98	03:38	300	151	149	52	135:57	255	222	33	119	26:06

Alerts: the number of bugs considered by the technique.
TP: the vulnerability trigger location is reachable by user-controllable data.
FP: the vulnerability trigger location is unreachable or not caused by user-controllable data.
Unique Bugs: different pairs of user-controllable data receiving locations and vulnerability triggering locations in TPs.
Avg. Time: average time spent analyzing a firmware represented by `hh:mm`.

analysis cannot be performed due to lack of sufficient context. This is due to the lack of associated code snippets that prevent syntax analysis such as dataflow and control flow. Regarding the task description granularity, LATTE performs better than the method without persona assignment and task splitting, with an average accuracy improvement of 2.78% and 6.4%, respectively. Compared to the granularity of code slicing, changes in the task description have a relatively small impact on the accuracy of the results. The main reason is that the LLM itself has sufficient understanding capability to reason and understand according to the task requirements, so persona assignment or task splitting does not have a significant impact on the results. However, providing richer context is still helpful for task understanding.

> *Answer to RQ4*: Compared with prompts (prompt sequences) constructed based on different code slicing granularitiesand analysis task description granularities,LATTE demonstrates higher accuracy in detecting binary vulnerabilities.

6.6 Real-World Vulnerability Inspection (RQ5)

RQ1–RQ4 evaluations demonstrated the vulnerability analysis capabilities of LATTE on standard datasets. To further illustrate LATTE's ability to analyze unknown and real-world data, we evaluated LATTE on 49 embedded device firmware samples from Karonte. LATTE issued a total of 255 alerts for 3,678 DFs. As shown in Table 6, the 255 alerts found after verification include 222 TPs of which there are 119 unique CWE-78 and CWE-120 bugs.

LATTE outperforms advanced binary taint analysis techniques. Karonte and Arbiter found 46 and 151 TPs, respectively, while LATTE found 222 TPs. Moreover, LATTE's precision 87.1% is higher than Karonte's 62.16% and Arbiter's 48.67%. For analysis time, Karonte and Arbiter rely on the heavyweight technique symbolic execution, so LATTE spends 425 and 109 minutes less time on analysis, respectively. Although Emtaint found 1,296 more TPs than LATTE, many of TPs were the same bugs caused by different dataflows. It takes a lot of work for analysts to verify the results.

Downloaded from the ACM Digital Library on April 7, 2025.

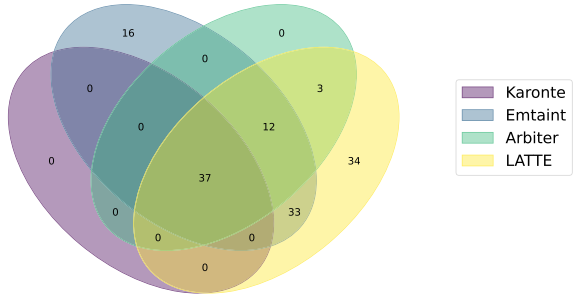


Fig. 16. Unique bugs discovered by different tools.

As shown in Figure 16, the unique bugs found by LATTE cover the results of Karonte and Arbiter, while 16 unique bugs found by Emtaint were not found by LATTE. The reason is that the pointer alias in LATTE is not accurate, resulting in no relationship between variables pointing to the same memory. Therefore, the CC extracted by LATTE is incomplete, causing the corresponding DF to be missed. However, LATTE found 119 unique bugs, 21 more than Emtaint. Moreover, LATTE found 37 previously unknown bugs, and 10 CVE numbers have been obtained due to the high threat.

> *Answer to RQ5*: LATTE effectively found 119 unique bugs (including 37 previously unknown bugs)in the real-world firmware dataset,outperforming the state-of-the-art techniques Karonte, Emtaint, and Arbiter.

7 Discussion

7.1 Limitations on Vulnerability Types

LATTE faces challenges in handling certain types of vulnerabilities, such as logic vulnerabilities [57], race condition vulnerabilities [58], and vulnerabilities involving complex mathematical calculations [56, 69]. These vulnerabilities often involve intricate code snippets with complex nesting and jumping, making them difficult for the LLM to effectively analyze. Therefore, the under/over-tainting problem is prone to occur. Moreover, these vulnerabilities are also difficult to detect by using taint analysis alone. Complex vulnerabilities require the extraction of more context information such as resource locks, interrupt vector tables, shared memory addresses, and so on to assist in analysis. Collecting more valuable context information and providing it to LLMs for vulnerability analysis is a direction worthy of further research.

7.2 Limitations on Binary Analysis

LATTE implements binary analysis based on Ghidra [62], including reverse engineering, function structure recovery, and calling relationship analysis. Ghidra provides relatively complete analysis algorithms and APIs for developers to use.

However, binary analysis itself is inherently complex and poses several challenges [52]. Many issues remain partially or entirely unsolved. For example, Ghidra's analysis can struggle to determine when two pointers refer to the same memory location, leading to potential inaccuracies in dataflow tracking and alias analysis. Additionally, Ghidra sometimes has difficulty resolving virtual calls, which can obscure the true calling relationships in the binary. This limitation is especially problematic in C++ programs with extensive use of polymorphism, where the inability to accurately resolve virtual calls can hinder the understanding of control flow and data dependencies. LLM

Downloaded from the ACM Digital Library on April 7, 2025.

excels at identifying patterns in code and dataflow, and it would be worth exploring further in the future to assist in identifying common indirect control flow patterns.

Moreover, when the compiler inlines a function, the original boundary between the caller function and the callee function is eliminated, making it difficult to reconstruct the original function structure and understand the precise calling relationship. Fortunately, if a callee is inlined, the caller becomes larger by incorporating the implementation of the callee. Therefore, it does not affect the analysis of the code with LLMs. Moreover, function inlining affects the identification of statically linked source and sink functions. But in most cases, the source and sink are provided by the system library [31, 88]. In order to ensure the size and compatibility of the program, the system library is usually dynamically linked, e.g., glibc [74, 88].

7.3 Importance of Metrics for Evaluating Vulnerability Analysis

Precision and recall are two key metrics for real program vulnerability analysis. Analytical methods often require a balance between FPs and FNs [88]. Precise analysis algorithms can reduce FPs, but may overlook diverse potential vulnerabilities, thus increasing FNs. Conversely, over-approximation analysis algorithms may identify more potential vulnerabilities, but also increase the number of FPs. Therefore, security engineers need to choose the appropriate method based on specific analysis contexts and objectives. Furthermore, identifying unique bugs is crucial for both precision and recall metrics. Duplicate TPs not only add no value to security analysis but also increase the overhead involved in validating vulnerabilities [12, 74].

The following guidelines provide tool selection recommendations based on analysis costs, business scenarios, and security requirement standards [36, 50, 61]. The recall score is particularly important when resources such as time and labor are sufficient. Additionally, using high-recall tools to find as many vulnerabilities as possible is especially useful in the early stages or when reviewing new code. On the other hand, when dealing with a large number of subjects and limited resources, security engineers should prioritize high-precision tools to ensure that reported vulnerabilities are genuine, thus minimizing the time spent on FPs. For systems requiring stringent security measures, use a combination of tools to balance precision and recall, ensuring comprehensive coverage and accurate identification of vulnerabilities.

According to the evaluation results in Section 6.2.1, LATTE outperforms other tools in both of precision and recall. Meanwhile, LATTE found more unique bugs in the evaluation of real firmwares in Section 6.6, and covered the bug results of Arbiter and Karonte. Based on the above guidelines, engineers should give priority to LATTE. However, the cost of LLM tokens for using LATTE is unavoidable. Therefore, if engineers have sufficient token resources, LATTE is recommended to be used over traditional methods.

7.4 Impact of Binary Protection on LATTE

LATTE performs binary analysis based on the intermediate representation P-Code of Ghidra, enabling binary analysis across different architectures such as X86, X64, ARM, and MIPS. Consistent with other binary vulnerability analysis techniques [10, 12, 74, 88], LATTE cannot directly analyze binaries that have been obfuscated or encrypted. Additional steps are needed to ensure the binary can be processed by Ghidra. There is ongoing research focused on deobfuscation and decryption, which can aid subsequent vulnerability analysis [25, 48, 86, 89].

Moreover, the performance of LATTE was evaluated on binaries lacking debugging information, symbol tables, and comments. Maliciously tampered semantics can interfere with LATTE's analysis. For example, renaming a function containing a vulnerability to "no_vulnerability" might increase the likelihood of misjudgment. However, by instructing LLMs to ignore such semantic information, the impact can be significantly reduced. Offensive and defensive techniques in binary analysis

Downloaded from the ACM Digital Library on April 7, 2025.

remain an ongoing challenge, and our findings contribute to understanding the capabilities and limitations of LLMs in this context.

7.5 Identification of Sources and Sinks

The specification of sources and sinks is necessary to initiate LATTE analysis. If the analyst already has the identified sources and sinks, the source and sink identification stage can be skipped and DF extraction and PS construction can be performed directly by setting the sources and sinks. If not, the analyst can use LATTE to help identify sources and sinks. We evaluated source and sink identification based on the Juliet test suite in Section 6.3. The identification results of sources and sinks can be reused if another binary uses the same dynamic library. We conducted vulnerability inspections using these identification results in real-world firmwares in Section 6.6.

In the real world, identifying sources and sinks is a complex problem. First, real programs require more queries due to the presence of various linked libraries. For example, the glibc requires about 3,000–4,000 queries, and the OpenGL requires several hundred queries. Therefore, even with LATTE to assist in identifying sources and sinks for caching, it requires a certain amount of time overhead. Second, as described in Section 7.2, although most binary programs use dynamic linking, static linking and inlining make it more difficult to identify sources and sinks. LATTE needs to be combined with program analysis techniques such as code summarization and dynamic analysis to further determine the source and sink. Third, some custom functions in real-world binaries may also be used as tainted sources and sinks [46], which further complicates the identification of sources and sinks. LLMs can recognize well-known functions like `recv` and `printf`, its performance is limited when encountering previously unseen functions. Meanwhile, the hallucination of LLMs seriously affects the identification of sources and sinks. Continuously refining and optimizing prompts or models based on expert feedback, code information, and specification data is critical to meeting the challenges of various libraries and custom functions.

7.6 Applicability to Source Code

While LATTE is designed for vulnerability analysis of binary code, its underlying concept is adaptable for source code analysis as well. If source code analysis is required, three components of the LATTE workflow (outlined in Figure 6) would need adjustment. First, source code analysis bypasses the need for disassembly or decompilation, allowing for direct code analysis to identify DFs. Second, the algorithm for extracting DFs must be modified to work with source code. For example, in decompiled code, the analysis relies on registers and memory addresses, whereas, in source code, it focuses on variable analysis. Furthermore, binary code typically integrates all calling relationships and dependencies into a single file, while source code requires parsing dependencies across multiple files. Third, the PTs need to be optimized. The start template in binary analysis states that the code is "generated by decompilation," which provides helpful context. When analyzing source code, removing this statement or indicating that the code is source code can help LLMs analyze the source code more directly, benefiting from explicit variable names and comments. Providing additional descriptive information further aids LLMs in analyzing the code effectively.

To demonstrate the feasibility of the concept of LATTE for source code, we utilized SVF [81] to extract DFs from the source code of the Juliet test cases (Section 6) and removed the "generated by decompilation" statement from the start prompt. As shown in Table 7, the accuracy of detecting vulnerabilities in source code for CWE-78, CWE-134, CWE-190, and CWE-606 improved by 1.64%, 2.91%, 17.9%, and 5.07%, respectively, compared to stripped binary analysis. The p-values for these comparisons, calculated using the Mann-Whitney U Test, were all less than `0.1`, indicating statistical significance. This improvement results from the semantic clarity in source code, with

Downloaded from the ACM Digital Library on April 7, 2025.

Table 7. Comparison of Vulnerability Checking Accuracy on Source Code and Decompiled Code

	Stripped Binary	Source Code	Improvement	p-Value
CWE-78	96.46%	98.1%	1.64%	0.095
CWE-134	93.88%	96.9%	3.02%	0.008
CWE-190	62.1%	80%	17.9%	0.008
CWE-606	73.33%	78.4%	5.07%	0.004

function names, variable names, and type definitions providing critical context that aids LLMs in understanding and analyzing vulnerabilities.

However, when combined with LLMs, there may be more effective approaches than LATTE for source code analysis. For instance, integrating LLMs with advanced program analysis tools such as CodeQL [34, 84, 90] could provide a more comprehensive understanding of code structure and semantics. Additionally, pointer analysis and other techniques could be applied directly to source code, rather than decompiled code, to further enhance the accuracy and effectiveness of vulnerability detection.

8 Related Work

Taint Analysis. Taint analysis is a software security analysis method that aims to identify and analyze potential security vulnerabilities in programs. It is widely used for analysis in source code, Web applications, and Android programs. VulSlicer [75] and FICS [2] performed dataflow analysis on the source and sink pairs in the source code. Merlin [47], Thomé et al. [85], and JOZA [60] summarized artificial experience into rules and used heuristic methods to discover taint-style vulnerability in Web applications. DroidSafe [30], SCANDAL [39], and AndroidLeaks [26] identified some API calls or callback functions as taint sources in Android programs to perform taint analysis. The main difference between taint analysis of binary programs and the above program types is the loss of information such as program semantics and code structure. Therefore, when implementing a binary taint analysis engine, more expert experience and more manual work are required. Specific binary taint analysis techniques are introduced in the next paragraph.

Binary Vulnerability Analysis. For vulnerability analysis of binary, there are both dynamic and static techniques. Dynamic techniques monitor unexpected behaviors of test targets as they execute test cases, such as fuzzing [8, 22, 44, 77, 103], concolic execution [15, 68, 80, 101], and emulation analysis [14, 21, 35, 40, 53]. The results of dynamic analysis are reliable, but the test code coverage is low and the test execution depends on the specific operating environment. Binary execution environments vary depending on the operating system, architecture, and even peripherals. As a result, the application and performance of dynamic analysis is often limited. Static analysis techniques are suitable for binary analysis due to their high code test coverage and not limited to the execution environment. BootStomp [73], Saluki [31], Dtaint [11], Karonte [74], SATC [10], Arbiter [88], and Emtaint [12] techniques primarily leverage taint analysis combined with program analysis, symbol execution, and machine learning to analyze binary vulnerabilities. However, these methods commonly rely on human experience to define propagation and inspection rules. LATTE, on the other hand, is the first to propose a taint analysis technique combined with LLMs for binary vulnerability inspection, offering a novel approach that avoids the need for manual rule definition.

LLM-Assisted Code Analysis. The LLM is a class of deep learning models that have significantly impacted natural language processing. In recent years, with the excellent code understanding ability of LLMs, they have been widely applied in code-related tasks, such as code generation

Downloaded from the ACM Digital Library on April 7, 2025.

[1, 24, 65, 100], patch generation [70, 79, 95, 96], and code summarization [3]. For code vulnerability analysis, it is often necessary to combine various program analysis techniques on the basis of LLMs' code understanding to conduct effective vulnerability auditing. For instance, FuzzGPT [18] and TitanFuzz [17] utilize the LLM to analyze existing code snippets and generate test cases to assist in API fuzzing. CODAMOSA [42] uses the LLM to analyze code coverage information and modify test cases to improve test coverage. Moreover, LLMDFA [91], inspired by summary-based dataflow analysis, introduced the dataflow analysis framework supported by LLM and explored the idea of LLM-assisted source code analysis. LLMSAN [92] further used the ideas of few-shot chain-of-thought and program properties to reduce the impact of LLM hallucination on the reliability of dataflow analysis. Unlike these LLM-assisted analysis techniques, LATTE utilizes the LLM to assist static analysis and works on binaries that provide less code information than source code.

Security Slicing. Obtaining code snippets from a program for vulnerability analysis is a popular security analysis method. To reduce the time overhead of verifying warnings and improve inspection accuracy, the results of code slicing are often combined with other analysis and testing techniques for further verification. FuzzSlice [59] and Helium [37] used static analysis technology to construct compilable harnesses from source code slices, which are then subjected to vulnerability verification through fuzzing. SFuzz [9] implemented fuzzing on independent code snippets extracted from real-time operating systems binaries. These techniques must ensure that the code snippets not only contain the entire input processing flow but also that the slicing result is compilable and executable. To address the constraints on the execution conditions of code snippets, Ramos and Engler [72] proposed under-constrained symbolic execution, allowing symbolic execution of any function without initializing any of its data structures or setting up any environment. However, this technique still faces challenges with loops, pointers, and library function calls, affecting analysis accuracy. Our approach, LATTE, leverages the idea of taint analysis to slice the program and then utilizes LLMs for vulnerability analysis. By using LLMs, our approach reduces the dependency on the execution environment, alleviating the need for comprehensive environment setup and initialization. LLMs provide a deeper semantic understanding of the code, improving the identification and analysis of vulnerabilities within the sliced snippets. Combining taint analysis with LLMs allows for a more focused and relevant slicing, ensuring that the critical parts of the code related to potential vulnerabilities are included in the analysis.

9 Conclusion

This article proposes LATTE, the first LLM-powered static binary taint analysis. Traditional taint analyzers rely on human expertise to manually define patterns and rules, and this manual nature severely limits the effectiveness and performance of the taint analysis technique. LATTE performs DF slicing by slicing VDs back to external input sources. LATTE then combines code snippets to construct PSs, leveraging LLMs' context-aware assistance and human-like understanding capabilities for code vulnerability inspection. Based on the substantial evaluations, LATTE outperforms the state of the art across all taint-style vulnerability types, such as buffer overflow and command injection. LATTE sheds light on the combination of program analysis and LLMs for vulnerability analysis.

Acknowledgment

We thank the anonymous reviewers for their valuable comments to improve our article.

References

[1] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2655–2668. Association for Computational Linguistics. Retrieved from https://www.aclweb.org/anthology/2021.naacl-main.211

Downloaded from the ACM Digital Library on April 7, 2025.

[2] Mansour Ahmadi, Reza Mirzazade farkhani, Ryan Williams, and Long Lu. 2021. Finding bugs using your own code: Detecting functionally-similar yet inconsistent code. In *30th USENIX Security Symposium (USENIX Security '21)*. USENIX Association, 2025–2040. Retrieved from https://www.usenix.org/conference/usenixsecurity21/presentation/ahmadi

[3] Toufique Ahmed and Premkumar Devanbu. 2023. Few-shot training LLMs for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. Association for Computing Machinery, New York, NY, Article 177, 1–5. DOI: https://doi.org/10.1145/3551349.3559555

[4] Manos Antonakakis, Tim April, Michael Bailey, Matt Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, and Michalis Kallitsis. 2017. Understanding the Mirai Botnet. In *26th USENIX Security Symposium*, 1093–1110.

[5] BlackDuck. 2024. Coverity Scan. Retrieved November 6, 2024 from https://scan.coverity.com/

[6] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. 2023. Sparks of artificial general intelligence: Early experiments with GPT-4. arXiv:2303.12712. Retrieved from https://arxiv.org/abs/2303.12712

[7] Marcel Busch, Aravind Machiry, Chad Spensky, Giovanni Vigna, Christopher Kruegel, and Mathias Payer. 2023. Teezz: Fuzzing trusted applications on cots Android devices. In *2023 IEEE Symposium on Security and Privacy (SP)*, 220–235.

[8] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Wang, Wing Cheong Lau, Menghan Sun, Ronghai Yang, and Kehuan Zhang. 2018. IoTFuzzer: Discovering memory corruptions in IoT through app-based fuzzing. In *Network and Distributed System Security Symposium (NDSS)*, 1–15.

[9] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, Yue Liu, Shanqing Guo, Haixin Duan, Kaida Jiang, et al. 2022. SFuzz: Slice-based fuzzing for real-time operating systems. In *2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. ACM, New York, NY, 485–498. DOI: https://doi.org/10.1145/3548606.3559367

[10] Libo Chen, Yanhao Wang, Quanpu Cai, Yunfan Zhan, Hong Hu, Jiaqi Linghu, Qinsheng Hou, Chao Zhang, Haixin Duan, and Zhi Xue. 2021. Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems. In *USENIX Security Symposium*, 303–319.

[11] Kai Cheng, Qiang Li, Lei Wang, Qian Chen, Yaowen Zheng, Limin Sun, and Zhenkai Liang. 2018. DTaint: Detecting the taint-style vulnerability in embedded device firmware. In *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 430–441.

[12] Kai Cheng, Yaowen Zheng, Tao Liu, Le Guan, Peng Liu, Hong Li, Hongsong Zhu, Kejiang Ye, and Limin Sun. 2023. Detecting vulnerabilities in Linux-based embedded firmware with SSE-based on-demand alias analysis. In *2023 ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 360–372.

[13] Jake Christensen, Ionut Mugurel Anghel, Rob Taglang, Mihai Chiroiu, and Radu Sion. 2020. DECAF: Automatic, adaptive de-bloating and hardening of COTS firmware. In *29th USENIX Security Symposium (USENIX Security '20)*. USENIX Association, 1713–1730. Retrieved from https://www.usenix.org/conference/usenixsecurity20/presentation/christensen

[14] Abraham A. Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, Giovanni Vigna, Saurabh Bagchi, and Mathias Payer. 2020. HALucinator: Firmware re-hosting through abstraction layer emulation. In *29th USENIX Security Symposium (USENIX Security '20)*, 1201–1218.

[15] Emilio Coppa, Heng Yin, and Camil Demetrescu. 2022. Symfusion: Hybrid instrumentation for concolic execution. In *37th IEEE/ACM International Conference on Automated Software Engineering*, 1–12.

[16] DAIR.AI. 2024. Prompt Engineering Guide. Retrieved June 1, 2024 from https://www.promptingguide.ai/techniques/fewshot

[17] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 423–435.

[18] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2024. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Association for Computing Machinery, New York, NY, Article 70, 1–13. DOI: https://doi.org/10.1145/3597503.3623343

[19] Edsger W. Dijkstra. 2002. *Ewd 1308: What Led to "Notes on Structured Programming."* Springer.

[20] Facebook. 2024. Infer. Retrieved November 6, 2024 from https://github.com/facebook/infer

[21] Bo Feng, Alejandro Mera, and Long Lu. 2020. P2IM: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling. In *29th USENIX Security Symposium (USENIX Security '20)*, 1237–1254.

[22] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, Surya Nepal, and Yang Xiang. 2021. Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference. In *2021 ACM SIGSAC Conference on Computer and Communications Security*, 337–350.

Downloaded from the ACM Digital Library on April 7, 2025.

[23] NSA Center for Assured Software. 2017. Juliet C/C++ 1.3. Retrieved May 1, 2023 from https://samate.nist.gov/SARD/test-suites/112
[24] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. 2022. Incoder: A generative model for code infilling and synthesis. arXiv:2204.05999. Retrieved from https://arxiv.org/abs/2204.05999
[25] Peter Garba and Matteo Favaro. 2019. Saturn-software deobfuscation framework based on LLVM. In *3rd ACM Workshop on Software Protection*, 27–38.
[26] Clint Gibler, Jonathan Crussell, Jeremy Erickson, and Hao Chen. 2012. Androidleaks: Automatically detecting potential privacy leaks in Android applications on a large scale. In *5th International Conference on Trust and Trustworthy Computing (TRUST '12)*. Springer, 291–307.
[27] Github. 2023. CodeQL. Retrieved May 1, 2023 from https://codeql.github.com/
[28] Github. 2023. OpenSSL. Retrieved May 1, 2023 from https://github.com/openssl/openssl
[29] Github. 2024. Joern. Retrieved May 12, 2024 from https://joern.io/
[30] Michael I. Gordon, Deokhwan Kim, Jeff H. Perkins, Limei Gilham, Nguyen Nguyen, and Martin C. Rinard. 2015. Information flow analysis of Android applications in Droidsafe. In *Network and Distributed System Security Symposium (NDSS)*, Vol. 15, 110.
[31] Ivan Gotovchits, Rijnard Van Tonder, and David Brumley. 2018. Saluki: Finding taint-style vulnerabilities with static property checking. In *NDSS Workshop on Binary Analysis Research*, Vol. 2018, 1–14.
[32] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of ChatGPT in automated code refinement: An empirical study. In *46th IEEE/ACM International Conference on Software Engineering*, 1–13.
[33] H. Han, J. Kyea, Y. Jin, J. Kang, B. Pak, and I. Yun. 2023. QueryX: Symbolic query on decompiled code for finding bugs in COTS binaries. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, 3279–3295. DOI: https://doi.org/10.1109/SP46215.2023.00064
[34] Jingxuan He and Martin Vechev. 2023. Large language models for code: Security hardening and adversarial testing. In *2023 ACM SIGSAC Conference on Computer and Communications Security*, 1865–1879.
[35] Grant Hernandez, Marius Muench, Dominik Maier, Alyssa Milburn, Shinjo Park, Tobias Scharnowski, Tyler Tucker, Patrick Traynor, and Kevin Butler. 2022. FIRMWIRE: Transparent dynamic analysis for cellular baseband firmware. In *Network and Distributed Systems Security Symposium (NDSS '22)*, 1–19.
[36] ISO. 2022. ISO/IEC 27001. Retrieved August 1, 2024 from https://www.iso.org/standard/27001
[37] Ashwin Kallingal Joshy, Xueyuan Chen, Benjamin Steenhoek, and Wei Le. 2021. Validating static warnings via testing code fragments. In *30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21)*. ACM, New York, NY, 540–552. DOI: https://doi.org/10.1145/3460319.3464832
[38] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
[39] Jinyung Kim, Yongho Yoon, Kwangkeun Yi, Junbum Shin, and SWRD Center. 2012. ScanDal: Static analyzer for detecting privacy leaks in Android applications. *MoST* 12, 110 (2012), 1.
[40] Mingeun Kim, Dongkwan Kim, Eunsoo Kim, Suryeon Kim, Yeongjin Jang, and Yongdae Kim. 2020. FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis. In *Annual Computer Security Applications Conference*, 733–745.
[41] R. Langner. 2011. Stuxnet: Dissecting a cyberwarfare weapon. *IEEE Security Privacy* 9, 3 (2011), 49–51.
[42] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *International Conference on Software Engineering (ICSE)*, 919–931.
[43] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2023. Assisting static analysis with large language models: A ChatGPT experiment. In *31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2107–2111.
[44] Wenqiang Li, Jiameng Shi, Fengjun Li, Jingqiang Lin, Wei Wang, and Le Guan. 2022. uAFL: Non-intrusive feedback-driven fuzzing for microcontroller firmware. In *44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 1–12. DOI: https://doi.org/10.1145/3510003.3510208
[45] Puzhuo Liu, Dongliang Fang, Chuan Qin, Kai Cheng, Shichao Lv, Hongsong Zhu, and Limin Sun. 2022. Finding vulnerabilities in internal-binary of firmware with clues. In *IEEE International Conference on Communications (ICC '22)*. IEEE, 5397–5402.
[46] Puzhuo Liu, Yaowen Zheng, Chengnian Sun, Chuan Qin, Dongliang Fang, Mingdong Liu, and Limin Sun. 2024. FITS: Inferring intermediate taint sources for effective vulnerability analysis of IoT device firmware. In *28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, Vol. 4. ACM, New York, NY, 138–152. DOI: https://doi.org/10.1145/3623278.3624759

Downloaded from the ACM Digital Library on April 7, 2025.

[47] Benjamin Livshits, Aditya V. Nori, Sriram K. Rajamani, and Anindya Banerjee. 2009. Merlin: Specification inference for explicit information flow problems. *SIGPLAN Notices* 44, 6 (Jun. 2009), 75–86. DOI: https://doi.org/10.1145/1543135.1542485

[48] Matias Madou, Ludo Van Put, and Koen De Bosschere. 2006. Loco: An interactive code (de) obfuscation tool. In *2006 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation*, 140–144.

[49] Abdalla Wasef Marashdih, Zarul Fitri Zaaba, and Khaled Suwais. 2023. An enhanced static taint analysis approach to detect input validation vulnerability. *Journal of King Saud University-Computer and Information Sciences* 35, 2 (2023), 682–701.

[50] Gary McGraw. 2006. *Software Security: Building Security In*. Addison-Wesley Professional. Retrieved from https://dl.acm.org/doi/book/10.5555/1121680

[51] Ruijie Meng, Martin Mirchev, Marcel Bohme, and Abhik Roychoudhury. 2024. Large language model guided protocol fuzzing. In *31st Annual Network and Distributed System Security Symposium (NDSS)*, 1–17.

[52] Xiaozhu Meng and Barton P. Miller. 2016. Binary code is not easy. In *25th International Symposium on Software Testing and Analysis*, 24–35.

[53] Alejandro Mera, Bo Feng, Long Lu, and Engin Kirda. 2021. DICE: Automatic emulation of DMA input channels for dynamic firmware analysis. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1938–1954.

[54] Mitre. 2023. Common Vulnerability Enumeration. Retrieved May 1, 2023 from https://cve.mitre.org/

[55] Mitre. 2023. Common Weakness Enumeration. Retrieved May 1, 2023 from https://cwe.mitre.org/

[56] Mitre. 2024. Cryptographic Issues. Retrieved August 1, 2024 from https://cwe.mitre.org/data/definitions/310.html

[57] Mitre. 2024. CWE CATEGORY: Business Logic Errors. Retrieved August 1, 2024 from https://cwe.mitre.org/data/definitions/840.html

[58] Mitre. 2024. Race Condition. Retrieved August 1, 2024 from https://cwe.mitre.org/data/definitions/362.html

[59] Aniruddhan Murali, Noble Mathews, Mahmoud Alfadel, Meiyappan Nagappan, and Meng Xu. 2024. FuzzSlice: Pruning false positives in static analysis warnings through function-level fuzzing. In *IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 65, 13 pages. DOI: https://doi.org/10.1145/3597503.3623321

[60] Abbas Naderi-Afooshteh, Anh Nguyen-Tuong, Mandana Bagheri-Marzijarani, Jason D. Hiser, and Jack W. Davidson. 2015. Joza: Hybrid taint inference for defeating web application SQL injection attacks. In *2015 45th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 172–183.

[61] NIST. 2020. NIST SP 800-53. Retrieved August 1, 2024 from https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final

[62] NSA. 2023. Ghidra. Retrieved May 1, 2023 from https://ghidra-sre.org

[63] OPENAI. 2023. GPT-4.0. Retrieved May 1, 2023 from hhttps://platform.openai.com

[64] OpenAI. 2023. Tokenizer. Retrieved May 1, 2023 from https://platform.openai.com/tokenizer

[65] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2022. Asleep at the keyboard? Assessing the security of Github Copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 754–768.

[66] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. 2023. Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. Association for Computing Machinery, New York, NY, 2785–2799. DOI: https://doi.org/10.1145/3576915.3623157

[67] Jannik Pewny, Behrad Garmany, Robert Gawlik, Christian Rossow, and Thorsten Holz. 2015. Cross-architecture bug search in binary executables. In *2015 IEEE Symposium on Security and Privacy*. IEEE, 709–724.

[68] Sebastian Poeplau and Aurélien Francillon. 2020. Symbolic execution with SymCC: Don't interpret, compile! In *29th USENIX Security Symposium (USENIX Security '20)*, 181–198.

[69] Marios Pomonis, Theofilos Petsios, Kangkook Jee, Michalis Polychronakis, and Angelos D. Keromytis. 2014. IntFlow: Improving the accuracy of arithmetic error detection using information flow tracking. In *30th Annual Computer Security Applications Conference (ACSAC '14)*. ACM, New York, NY, 416–425. DOI: https://doi.org/10.1145/2664243.2664282

[70] Julian Aron Prenner, Hlib Babii, and Romain Robbes. 2022. Can OpenAI's codex fix bugs? An evaluation on QuixBugs. In *3rd International Workshop on Automated Program Repair*, 69–75.

[71] Rui Qiao and R. Sekar. 2017. Function interface analysis: A principled approach for function recognition in COTS binaries. In *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 201–212.

[72] David A. Ramos and Dawson Engler. 2015. Under-constrained symbolic execution: Correctness checking for real code. In *24th USENIX Security Symposium (USENIX Security '15)*. USENIX Association, Washington, D.C., 49–64. Retrieved from https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/ramos

[73] Nilo Redini, Aravind Machiry, Dipanjan Das, Yanick Fratantonio, Antonio Bianchi, Eric Gustafson, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2017. BootStomp: On the security of bootloaders in mobile devices. In *26th USENIX Security Symposium (USENIX Security '17)*, 781–798.

Downloaded from the ACM Digital Library on April 7, 2025.

[74] Nilo Redini, Aravind Machiry, Ruoyu Wang, Chad Spensky, Andrea Continella, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2020. Karonte: Detecting insecure multi-binary interactions in embedded firmware. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1544–1561.

[75] Solmaz Salimi and Mehdi Kharrazi. 2022. VulSlicer: Vulnerability detection through code slicing. *Journal of Systems and Software* 193 (2022), 111450. DOI: https://doi.org/10.1016/j.jss.2022.111450

[76] Haziq Bin Sanib, Tan Joven, Tee Kai Guan, Sree Vathsan, and Toh Zheng Hong Shawn. 2022. Ghost. Retrieved May 1, 2023 from https://github.com/tthtlc/source-sink-ghidra

[77] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, Christopher Kruegel, Thorsten Holz, and Ali Abbasi. 2022. Fuzzware: Using precise MMIO modeling for effective firmware fuzzing. In *31st USENIX Security Symposium (USENIX Security '22)*, 1239–1256.

[78] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, et al. 2016. SOK: (State of) the art of war: Offensive techniques in binary analysis. In *2016 IEEE Symposium on Security and Privacy (SP)*, 138–157. DOI: https://doi.org/10.1109/SP.2016.17

[79] Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An analysis of the automatic bug fixing performance of ChatGPT. arXiv:2301.08653. Retrieved from https://arxiv.org/abs/2301.08653

[80] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2016. Driller: Augmenting fuzzing through selective symbolic execution. In *Network and Distributed System Security Symposium (NDSS)*, Vol. 16, 1–16.

[81] Yulei Sui. 2024. Static Value-Flow Analysis Framework for Source Code. Retrieved May 1, 2024 from https://github.com/SVF-tools/SVF

[82] Weisong Sun, Chunrong Fang, Yudu You, Yun Miao, Yi Liu, Yuekang Li, Gelei Deng, Shenghan Huang, Yuchen Chen, Quanjun Zhang, et al. 2023. Automatic code summarization via ChatGPT: How far are we? arXiv:2305.12865. Retrieved from https://arxiv.org/abs/2305.12865

[83] Joshua Henrina Sundjaja, Rijen Shrestha, and Kewal Krishan. 2023. McNemar and Mann-Whitney U Tests. Retrieved August 1, 2024 from https://www.ncbi.nlm.nih.gov/books/NBK560699/

[84] Nigar M. Shafiq Surameery and Mohammed Y. Shakor. 2023. Use ChatGPT to solve programming bugs. *International Journal of Information Technology & Computer Engineering* 1 (2023), 17–22.

[85] Julian Thomé, Lwin Khin Shar, Domenico Bianculli, and Lionel Briand. 2018. Security slicing for auditing common injection vulnerabilities. *Journal of Systems and Software* 137 (2018), 766–783. DOI: https://doi.org/10.1016/j.jss.2017.02.040

[86] Ramtine Tofighi-Shirazi, Irina-Mariuca Asavoae, Philippe Elbaz-Vincent, and Thanh-Ha Le. 2019. Defeating opaque predicates statically through machine learning and binary analysis. In *3rd ACM Workshop on Software Protection*, 3–14.

[87] Omer Tripp, Marco Pistoia, Stephen J. Fink, Manu Sridharan, and Omri Weisman. 2009. TAJ: Effective taint analysis of web applications. *SIGPLAN Notices* 44, 6 (Jun. 2009), 87–97. DOI: https://doi.org/10.1145/1543135.1542486

[88] Jayakrishna Vadayath, Moritz Eckert, Kyle Zeng, Nicolaas Weideman, Gokulkrishna Praveen Menon, Yanick Fratantonio, Davide Balzarotti, Adam Doupe, Tiffany Bao, Ruoyu Wang, et al. 2022. Arbiter: Bridging the static and dynamic divide in vulnerability discovery on binary programs. In *31st USENIX Security Symposium (USENIX Security '22)*, 413–430.

[89] Aurélien Vasselle, Philippe Maurine, and Maxime Cozzi. 2019. Breaking mobile firmware encryption through near-field side-channel analysis. In *3rd ACM Workshop on Attacks and Solutions in Hardware Security Workshop*, 23–32.

[90] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. 2023. Frustrated with code quality issues? LLMs can help! arXiv:2309.12938. Retrieved from https://arxiv.org/abs/2309.12938

[91] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, Xiaoheng Xie, and Xiangyu Zhang. 2024. LLMDFA: Analyzing dataflow in code with large language models. In *Annual Conference on Neural Information Processing Systems*, 1–24.

[92] Chengpeng Wang, Wuqi Zhang, Zian Su, Xiangzhe Xu, and Xiangyu Zhang. 2024. Sanitizing large language models in bug detection with data-flow. In *the 2024 Conference on Empirical Methods in Natural Language Processing*, 1–16.

[93] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with ChatGPT. arXiv:2302.11382. Retrieved from https://arxiv.org/abs/2302.11382

[94] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How effective are neural networks for fixing security vulnerabilities. In *32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. ACM, New York, NY, 1282–1294. DOI: https://doi.org/10.1145/3597926.3598135

Downloaded from the ACM Digital Library on April 7, 2025.

[95] Chunqiu Steven Xia and Lingming Zhang. 2022. Less training, more repairing please: Revisiting automated program repair via zero-shot learning. In *30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 959–971.

[96] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational automated program repair. arXiv:2301.13246. Retrieved from https://arxiv.org/abs/2301.13246

[97] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024)*. Association for Computing Machinery, New York, NY, 819–831. DOI: https://doi.org/10.1145/3650212.3680323

[98] Fabian Yamaguchi, Alwin Maier, Hugo Gascon, and Konrad Rieck. 2015. Automatic inference of search patterns for taint-style vulnerabilities. In *2015 IEEE Symposium on Security and Privacy*. IEEE, 797–812.

[99] Burak Yetiştiren, Işik Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub copilot, Amazon CodeWhisperer, and ChatGPT. arXiv:2304.10778. Retrieved from https://arxiv.org/abs/2304.10778

[100] Burak Yetistiren, Isik Ozsoy, and Eray Tuzun. 2022. Assessing the quality of GitHub Copilot's code generation. In *18th International Conference on Predictive Models and Data Analytics in Software Engineering*, 62–71.

[101] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, and Taesoo Kim. 2018. QSYM: A practical concolic execution engine tailored for hybrid fuzzing. In *27th USENIX Security Symposium (USENIX Security '18)*, 745–761.

[102] Zhuo Zhang, Yapeng Ye, Wei You, Guanhong Tao, Wen-chuan Lee, Yonghwi Kwon, Yousra Aafer, and Xiangyu Zhang. 2021. Osprey: Recovery of variable and data structure via probabilistic analysis for stripped binary. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 813–832.

[103] Yaowen Zheng, Ali Davanian, Heng Yin, Chengyu Song, Hongsong Zhu, and Limin Sun. 2019. FIRM-AFL: High-throughput greybox fuzzing of IoT firmware via augmented process emulation. In *28th USENIX Security Symposium (USENIX Security '19)*, 1099–1114.

Received 3 November 2023; revised 8 November 2024; accepted 11 December 2024

Downloaded from the ACM Digital Library on April 7, 2025.